Blind Signatures from Oblivious Transfer

Abstract.

1 Preliminaries

Notations. We denote the security parameter by λ. A polynomial time (PT) algorithm $\mathcal{A}$ runs in time polynomial in the (implicit) security parameter λ. We denote "probabilistic polynomial time" by PPT. We write $\mathsf{Time}(\mathcal{A})$ for the runtime of $\mathcal{A}$. A function $f(\lambda)$ is *negligible* in λ if it is $\mathcal{O}(\lambda^{-c})$ for every $c \in \mathbb{N}$. We write $f = \mathrm{negl}(\lambda)$ for short. Similarly, we write $f = \mathsf{poly}(\lambda)$ if $f(\lambda)$ is a polynomial with variable λ. If D is a probability distribution, $x \leftarrow D$ means that x is sampled from D and if S is a set, $x \leftarrow S$ means that x is sampled uniformly and independently at random from S. We also write $|S|$ for the cardinality of set S. Further, we write $D_0 \overset{c}{\approx} D_1$ for distributions D_0, D_1, if for all PPT adversaries $\mathcal{A}$, we have $|\Pr[x_0 \leftarrow D_0 : \mathcal{A}(1^\lambda, x_0) = 1] - \Pr[x_1 \leftarrow D_1 : \mathcal{A}(1^\lambda, x_1) = 1]| = \mathrm{negl}(\lambda)$. Similarly, we write $D_0 \overset{s}{\approx} D_1$ if the above holds even for unbounded adversaries. For some PPT algorithm $\mathcal{A}$, we write $\mathcal{A}^{\mathcal{O}}$ if $\mathcal{A}$ has oracle access to the oracle $\mathcal{O}$. If $\mathcal{A}$ performs some check, and the check fails, we assume that $\mathcal{A}$ outputs $\perp$ immediately. Generally, we assume that adversaries are implicitly stateful. We denote with $[n]$ the set $\{1, \ldots, n\}$ for $n \in \mathbb{N}$. We write $\mathbb{P}$ for the set of primes and $\mathbb{P}_I$ for the set of primes in the interval I. For some odd prime p, we use the representatives $\{-\frac{p-1}{2}, \cdots, \frac{p-1}{2}\}$ for $\mathbb{Z}_p$. For a group $\mathbb{G}$ we write $\mathsf{ord}(\mathbb{G})$ to denote the order of $\mathbb{G}$ and unless stated otherwise we write $\mathbb{G}$ with additive notation. For a group element g we write $\mathsf{ord}(g)$ to denote the order of the group element. We denote by $\mathsf{QR}_N = \{a \in \mathbb{Z}_N^* : \exists\, b \in \mathbb{Z}_N^*, b^2 \equiv a \bmod N\}$ the quadratic residues $\bmod N$. For some $N \in \mathbb{N}$, the group QR_N is a cyclic subgroup of $\mathbb{Z}_N^*$ and we denote by $\mathsf{Gen}(\mathsf{QR}_N)$ the set of generators of QR_N. Some properties of QR_N are recalled in lemma 1 in Section 2.

Probability. Let $V, L \in \mathbb{N}$. We define *uniform rejection sampling* for the interval $[V, (V+1)L]$ with masking overhead L as in [5]. Let $v \in [0, V]$. To mask v additively with a mask μ via rejection sampling, perform the following steps.

1. Draw a random mask $\mu \leftarrow [0, (V+1)L]$.
2. Abort if $v + \mu \notin [V, (V+1)L]$.
3. Output $w = v + \mu$.

The value w is uniform over $[V, (V+1)L]$ conditioned on no abort and the abort probability is at most $1/L$[1]. We use a version of the Forking Lemma from [1, Lemma 1] that fits our usage of it. The lemma was first introduced by Pointcheval and Stern [14] then generalized in [4, 1]. The formal statement can be found in Section 2.2.

Hardness Asssumptions. We use the following assumptions in this paper. Let GenG be a PPT algorithm that on input 1^λ and prime order p, outputs (a description of) a group $\mathbb{G} \leftarrow \mathsf{GenG}(1^\lambda)$ of order p. We generally use additive notations for prime order groups and capital letters for elements. Also, we assume that given the description, group operations and membership tests are efficient. We write $g \leftarrow \mathbb{G}$ for drawing elements from some group $\mathbb{G}$ at random. In the following, we assume that prime order groups are setup with GenG implicitly.

Let GenRSA be a PPT algorithm that on input 1^λ outputs $(N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda)$ such that $N = P \cdot Q$ with $P, Q \in \mathbb{P}$, where $P = 2P' + 1$ and $Q = 2Q' + 1$ are strong primes (*i.e.*, P', Q' are also primes). We assume that $P', Q' > 2^{\lambda+1}$.

First of all, the (D, ℓ)-*relaxed DLOG assumption with regards to* $\vec{g}$, where $\vec{g} = (g_0, \ldots, g_\ell) \in \mathsf{QR}_N^{\ell+1}$, assumes that for any PPT adversary, given $(N, g_0, \ldots, g_\ell)$ it is only with negligible probability

[1] When $v = 0$, the number of "bad" $\mu \in [0, (V+1)L]$ causing abort is $\#\{[0, V-1]\} = V$. For a fixed $0 < v \in (0, V]$, the number of "bad" μ causing abort is $\#\{[0, V-1-v]\} + \#\{[(V+1)L - v + 1, (V+1)L]\} = V$. In both cases the abort probability over choices of μ is $V/((V+1)L+1) \leq 1/L$.

to output $(c, d, x_0, \ldots, x_\ell)$ satisfying $c^d = \prod_{i=0}^{\ell} g_i^{x_i} \wedge \exists i\colon \frac{x_i}{d} \notin \mathbb{Z} \wedge d \in [0, D] \wedge x_i \in \mathbb{Z}$. The (D, ℓ)-relaxed DLOG assumption holds under the strong RSA assumption for all $D \leq 2^{\lambda+1}$. Next, the *Decisional Diffie-Hellman* (DDH) assumption in a cyclic group $\mathbb{G}$ assumes that for all PPT adversary it is only with negligible probability that the adversary can distinguish (aG, bG, abG) from (aG, bG, cG) where $G \leftarrow \mathbb{G}$ and $a, b, c \leftarrow \mathsf{ord}(\mathbb{G})$. Finally, the *strong* RSA (sRSA) assumes that it is only with negligible probability for any PPT adversary to output (e, z) such that $z^e \equiv y \bmod N$.

Explaining Random Group Elements as Random Strings. For our framework, we require commitments with uniform public parameters pp. For readability, we allow pp (and also uniform random strings urs of NIZKs) to contain (uniform) group elements g of prime-order groups $\mathbb{G}$ with known order p. This is without loss of generality because with *explainable sampling*, we can explain $g \leftarrow \mathbb{G}$ as a random bitstring. We refer to, *e.g.*, [12, Appendix B] for more details.

1.1 Cryptographic Primitives

Commitment Scheme. A *commitment scheme* is a tuple of PPT algorithms $\mathsf{C} = (\mathsf{C.Setup}, \mathsf{C.Commit}, \mathsf{C.Commit})$ such that

- $\mathsf{C.Setup}(1^\lambda)$: generates the public parameters pp,
- $\mathsf{C.Commit}(\mathsf{pp}, m)$: given the public parameters pp, message $m \in \mathcal{C}_{\mathsf{msg}}$, computes a commitment $c \in \mathcal{C}_{\mathsf{com}}$ with opening randomness d, and outputs the pair (c, d),
- $\mathsf{Verify}(\mathsf{pp}, c, m, d)$: given the public parameters pp, message $m \in \mathcal{C}_{\mathsf{msg}}$, and opening randomness d, outputs a bit $b \in \{0, 1\}$ which depends on the validity of the opening (m, d) with respect to the commitment c.

Here, $\mathcal{C}_{\mathsf{msg}}$, $\mathcal{C}_{\mathsf{rnd}}$, $\mathcal{C}_{\mathsf{com}}$, are message, randomness, and commitment spaces, respectively. If the public parameters are uniform or explainable (*i.e.*, Setup outputs some $\mathsf{pp} \leftarrow \{0, 1\}^\ell$ for $\ell \in \mathbb{N}$) we omit Setup without loss of generality.

We require the correctness, hiding and binding properties for a commitment scheme. A commitment schemes is *correct*, if honest commitments $(c, d) \leftarrow \mathsf{Commit}(\mathsf{pp}, m; r)$ always verify, *i.e.* it holds that $\mathsf{Verify}(\mathsf{pp}, c, m, d) = 1$ where pp are the public parameters. It is *hiding* if it is hard to decide whether an unopened commitment c commits to message m_0 or m_1, and it is *binding* if it is hard to open commitments c to distinct messages. We can have *computational, statistical, perfect* variants for hiding and binding properties. The formal definitions can be found in Section 2.5.

(Bounded) Integer Commitments. We refer to a commitment scheme with message space $[\mathbb{A}, B] \subseteq \mathbb{N}$ as a (bounded) integer commitment scheme. We often omit the term bounded if the message space is clear by context.

ElGamal commitments. We recall ElGamal (EG) over a group $\mathbb{G}$ of prime order p with message space $\mathbb{Z}_p$ [7]. We use additive notation for prime order groups.

- $\mathsf{EG.GenPP}(1^\lambda)$: set $(G, H) \leftarrow \mathbb{G}$ and output $\mathsf{pp} = (G, H)$.
- $\mathsf{EG.Commit}(\mathsf{pp}, m)$: sample $r \leftarrow \mathbb{Z}_p$ and set $c = (mG + rH, rG)$, and output (c, r).
- $\mathsf{EG.Verify}(\mathsf{pp}, c, m, r)$: check if $c = (mG + rH, rG)$.

Note that the public parameters are uniform and we can sample them via a random oracle to avoid trusted setup. EG commitments are correct, hiding under DDH and perfectly binding.

Remark 1. If in verification of EG, we check that $m \in [0, M]$ for $M < p$, then m is fixed over the integers and we can interpret the commitment as an integer commitment with message space $[0, M] \subseteq \mathbb{N}$.

Pedersen Commitments in $\mathbf{QR}_N$. We recall Pedersen multi-commitments (MPed) over QR_N with message space $\mathbb{Z}^\ell$ for some $\ell \in \mathbb{N}$ (cf. [6]).

- $\mathsf{MPed.GenPP}(1^\lambda)$: set $(N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda)$ and sample ℓ random generators g_i of QR_N, and output $\mathsf{pp} = (N, h, g_1, \cdots, g_\ell)$. Note that with (P, Q), we can check whether g_i generates QR_N.
- $\mathsf{MPed.Commit}(\mathsf{pp}, \vec{m})$: sample $r \leftarrow [0, N \cdot 2^\lambda]$, set $c \leftarrow h^r \cdot \prod_{i=1}^{\ell} g_i^{m_i} \bmod N$, and output (c, r).
- $\mathsf{MPed.Verify}(\mathsf{pp}, c, \vec{m}, r)$: check if $c = \pm h^r \cdot \prod_{i=1}^{\ell} g_i^{m_i} \bmod N$.

MPed commitments are correct, statistically hiding and binding under the factoring assumption (which is implied by sRSA). Throughout this work, we use MPed commitments in QR_N to enforce in security proofs that values extracted from NIZKs are integers via lemma 4.

Signature Scheme. A signature scheme is a tuple of PPT algorithms $\mathsf{S} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ such that

- $\mathsf{KeyGen}(1^\lambda)$: generates a verification key vk and a signing key sk,
- $\mathsf{Sign}(\mathsf{sk}, m)$: given a signing key sk and a message $m \in \mathcal{S}_{\mathsf{msg}}$, *deterministically* outputs a signature σ,
- $\mathsf{Verify}(\mathsf{vk}, m, \sigma)$: given a verification key pk and a signature σ on message m, *deterministically* outputs a bit $b \in \{0, 1\}$.

Here, $\mathcal{S}_{\mathsf{msg}}$ is the message space. We define the standard notion of *correctness* and `euf-cma` security. Correctness requires that any honestly generated signature $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ verifies, *i.e.* $\mathsf{Verify}(\mathsf{vk}, m, \sigma) = 1$. The `euf-cma` security imposes that even with oracle accesses to $\mathsf{Sign}(\mathsf{sk}, \cdot)$, no PPT adversary will be able to forge a valid signature σ on a message m that is not queried to $\mathsf{Sign}(\mathsf{sk}, \cdot)$.

Blind Signature Scheme. A (two-move) blind signature scheme is a tuple of PPT algorithms $\mathsf{BS} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ such that

Definition 1 (Blind Signature). *A (two-move)* blind signature scheme *is a tuple of PPT algorithms* $\mathsf{PBS} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *such that*

- $\mathsf{BSGen}(1^\lambda)$*: generates the verification key* bvk *and signing key* bsk*,*
- Sign *is split into the following algorithms:*
  - $\mathsf{BSUser}(\mathsf{bvk}, m)$*: given verification key* bvk *and message* $m \in \mathcal{BS}_{msg}$*, outputs a first message* ρ_1 *and a state* st*,*
  - $\mathsf{BSSigner}(\mathsf{bsk}, \rho_1)$*: given signing key* bsk *and first message* ρ_1*, outputs a second message* ρ_2*,*
  - $\mathsf{BSDerive}(\mathsf{st}, \rho_2)$*: given state* st *and second message* ρ_2*, outputs a signature* σ
- $\mathsf{BSVerify}(\mathsf{bvk}, m, \sigma)$*: given verification key* bvk *and signature* σ *on message* $m \in \mathcal{BS}_{msg}$*, outputs a bit* $b \in \{0, 1\}$*.*

Here, $\mathcal{BS}_{msg}$ *is the message spaces.*

We consider the standard security notions for blind signatures [11]. Below, we define correctness, blindness under *malicious keys*, and one-more unforgeability of a blind signature scheme. Moreover, we will omit the state for better readability on occasion.

Definition 2 (Correctness). *A blind signature scheme is* correct*, if for all messages* $m \in \mathcal{BS}_{msg}$*,* $(\mathsf{bvk}, \mathsf{bsk}) \leftarrow \mathsf{BSGen}(1^\lambda), (\rho_1, \mathsf{st}) \leftarrow \mathsf{BSUser}(\mathsf{bvk}, m), \rho_2 \leftarrow \mathsf{BSSigner}(\mathsf{bsk}, \rho_1), \sigma \leftarrow \mathsf{BSDerive}(\mathsf{st}, \rho_2)$*, it holds that* $\mathsf{BSVerify}(\mathsf{bvk}, m, \sigma) = 1$*.*

Definition 3 (Blindness Under Malicious Keys). *A blind signature scheme is* blind under malicious keys *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{blind}}_{\mathcal{A}}(\lambda) = \left| \Pr \left[\begin{array}{l} (\mathsf{bvk}, m_0, m_1) \leftarrow \mathcal{A}(1^\lambda),\ \mathsf{coin} \leftarrow \{0,1\}, \\ (\rho_{1,b}, \mathsf{st}_b) \leftarrow \mathsf{BSUser}(\mathsf{bvk}, m_b) \text{ for } b \in \\ \{0,1\}, \\ (\rho_{2,\mathsf{coin}}, \rho_{2,1-\mathsf{coin}}) \leftarrow \mathcal{A}(\rho_{1,\mathsf{coin}}, \rho_{1,1-\mathsf{coin}}),\ : \mathsf{coin} = \mathcal{A}(\sigma_0, \sigma_1) \\ \sigma_b \leftarrow \mathsf{BSDerive}(\mathsf{st}_b, \rho_{2,b}) \text{ for } b \in \{0,1\}, \\ \text{if } \exists b \text{ s.t. } \mathsf{BSVerify}(\mathsf{bvk}, m_b, \sigma_b) = 0\text{:} \\ \quad \text{then } \sigma_0 = \sigma_1 = \bot, \end{array} \right] - \frac{1}{2} \right| = \mathrm{negl}(\lambda).$$

Definition 4 (One-more Unforgeability). *A blind signature scheme is* one-more unforgeable *if for any $Q = \mathsf{poly}(\lambda)$ and PPT adversary $\mathcal{A}$ that makes at most Q signing queries, we have*

$$\mathsf{Adv}^{\mathrm{omuf}}_{\mathcal{A}}(\lambda) = \Pr\left[\begin{array}{l|l}(\mathsf{bvk},\mathsf{bsk}) \leftarrow \mathsf{BSGen}(1^\lambda) & \\ \{(m_i,\sigma_i)\}_{i\in[Q+1]} & \leftarrow : \begin{array}{l}\forall i \neq j \in [Q+1]: \\ m_i \neq m_j \\ \wedge \\ \mathsf{BSVerify}(\mathsf{bvk}, m_i, \sigma_i) = 1\end{array} \\ \mathcal{A}^{\mathsf{BSSigner}(\mathsf{bsk},\cdot)}(\mathsf{bvk}) & \end{array}\right] = \mathrm{negl}(\lambda).$$

Σ-Protocol. Let R be an NP relation with statements x and witnesses w. We denote by $\mathscr{L}_{\mathsf{R}} = \{x \mid \exists w \text{ s.t. } (x,w) \in \mathsf{R}\}$ the language induced by R. A Σ-protocol for an NP relation R for language $\mathscr{L}_{\mathsf{R}}$ with challenge space $\mathcal{CH}$ is a tuple of PPT algorithms $\Sigma = (\mathsf{Init}, \mathsf{Chall}, \mathsf{Resp}, \mathsf{Verify})$ such that

- $\mathsf{Init}(x,w)$: given a statement $x \in \mathscr{L}_{\mathsf{R}}$, and a witness w such that $(x,w) \in \mathsf{R}$, outputs a first flow message (*i.e.*, commitment) Ω and a state st, where we assume st includes x, w,
- $\mathsf{Chall}()$: samples a challenge $\gamma \leftarrow \mathcal{CH}$ (without taking any input),
- $\mathsf{Resp}(\mathsf{st},\gamma)$: given a state st and a challenge $\gamma \in \mathcal{CH}$, outputs a third flow message (*i.e.*, response) τ,
- $\mathsf{Verify}(x,\Omega,\gamma,\tau)$: given a statement $x \in \mathscr{L}_{\mathsf{R}}$, a commitment Ω, a challenge $\gamma \in \mathcal{CH}$, and a response τ, outputs a bit $b \in \{0,1\}$.

We recall the standard notions of *correctness, high-min entropy, (non-abort) honest-verifier zero-knowledge*, and *k-special soundness*. A Σ-protocol is correct, if for all $(x,w) \in \mathsf{R}$, if for any honestly generated transcripts (Ω,γ,τ), the verifier accepts, *i.e.* $\mathsf{Verify}(x,\Omega,\gamma,\tau) = 1$. It has high min-entropy if for all $(x,w) \in \mathsf{R}$, it is *statistically* hard to predict a honestly generated first flow Ω. It is honest-verifier zero-knowledge (HVZK), if there exists a PPT zero-knowledge simulator Sim such that the distributions of $\mathsf{Sim}(x,\gamma)$ and a honestly generated *non-aborting* transcript with Init initialized with (x,w) are statistically indistinguishable for any $x \in \mathscr{L}_{\mathsf{R}}$, and $\gamma \in \mathcal{CH}$, where the honest execution is conditioned on γ being used as the challenge. Finally, it is k-special sound, if there exists a *deterministic* PT extractor Ext such that given k valid transcripts $\{(\Omega,\gamma_i,\tau_i)\}_{i\in[k]}$ for statement x with pairwise distinct challenges $(\gamma_i)_i$, outputs a witness w such that $(x,w) \in \mathsf{R}$.

Non-Interactive Zero Knowledge. All formal definitions of the following can be found in Section 2.9. Let $\mathcal{URS} = \{0,1\}^\ell$ be a set of *uniform random strings* for some $\ell \in \mathbb{N}$ and $\mathcal{SRS}$ be some set of *structured random strings* with efficient membership test [2]. A NIZK for a relation R with common reference string space $\mathcal{CRS} = \mathcal{SRS} \times \mathcal{URS}$ is a tuple of PPT algorithms $(\mathsf{GenSRS}, \mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$, where the latter two are oracle-calling, such that:

- $\mathsf{GenSRS}(1^\lambda)$: outputs a structured reference string $\mathsf{srs} \in \mathcal{SRS}$,
- $\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs},x,w)$: receives a $\mathsf{crs} = (\mathsf{srs},\mathsf{urs}) \in \mathcal{CRS}$, a statement x and a witness w, and outputs a proof π,
- $\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs},x,\pi)$: receives a $\mathsf{crs} = (\mathsf{srs},\mathsf{urs}) \in \mathcal{CRS}$, a statement x and a proof π, and outputs a bit $b \in \{0,1\}$.

We recall that $\mathscr{L}_{\mathsf{R}} = \{x \mid \exists w \ : \ (x,w) \in \mathsf{R}\}$ denotes the language induced by R. If there is no crs needed, *i.e.* $\mathcal{CRS} = \varnothing$, we then omit crs as an input to Prove and Verify. A NIZK is *correct* if for any $\mathsf{crs} = (\mathsf{srs},\mathsf{urs})$ with $\mathsf{srs} \leftarrow \mathsf{GenSRS}(1^\lambda)$ and $\mathsf{urs} \leftarrow \mathcal{URS}$, $(x,w) \in \mathsf{R}$, and $\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathsf{crs},x,w)$, it holds that $\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs},x,\pi) = 1$. It is *zero-knowledge* if there exists a PPT simulator $\mathsf{Sim} = (\mathsf{Sim}_{\mathsf{crs}}, \mathsf{Sim}_{\mathsf{H}}, \mathsf{Sim}_{\pi})$ such that the distributions of $\pi' \leftarrow \mathsf{Sim}_{\pi}(\mathsf{crs},x)$ and $\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathsf{crs},x,w)$ are computationally indistinguishable for any $(x,w) \in \mathsf{R}$. Note that the sub-algorithms of Sim share state. For simulated proofs, the algorithm $\mathsf{Sim}_{\mathsf{H}}$ simulates the random oracle and $\mathsf{Sim}_{\mathsf{crs}}$ simulates the $\mathsf{crs} = (\mathsf{srs},\mathsf{urs})$, where there is an structured part srs. We also define a notion of *subversion zero-knowledge*, inspired by the notion introduced in [3]. To recall, the second part of the $\mathsf{crs} = (\mathsf{srs},\mathsf{urs})$ is a random reference string which can later be sampled via a random oracle, and the first part is a structured string srs. For *subversion zero-knowledge*, there is no $\mathsf{Sim}_{\mathsf{crs}}$

[2] This membership test is required for our definition of subversion zero-knowledge. Let H be a random oracle. Note that in general it is difficult to check that some srs was generated via GenSRS. (We allow that $\mathcal{SRS}$ is not equal to the output space of GenSRS.)

anymore and the structured srs can be chosen by $\mathcal{A}$, while urs is sampled uniformly at random by H for the real proofs $\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, x, w)$ or by $\mathsf{Sim}_{\mathsf{H}}$ for the simulated proofs $\pi' \leftarrow \mathsf{Sim}_{\pi}(\mathsf{crs}, x)$. Here we also require that the subverted srs belongs to $\mathcal{SRS}$.

We define *adaptive knowledge soundness*. An NIZK is adaptively knowledge sound for relation[3] $\widetilde{\mathsf{R}}$ if there exist positive polynomials $p_{\mathsf{T}}, p_{\mathsf{P}}$, constant c, a PPT extractor Ext and a PPT simulator SimCRS so that for any $(\overline{\mathsf{crs}}, \mathsf{td}) \leftarrow \mathsf{SimCRS}(1^\lambda)$, given oracle access to any PPT $\mathcal{A}$ (with explicit random tape ρ and making $Q_{\mathsf{H}} = \mathsf{poly}(\lambda)$ RO queries) that cannot distinguish $\overline{\mathsf{crs}} \in \mathcal{CRS}$ from a real $\mathsf{crs} := (\mathsf{srs} \leftarrow \mathsf{GenSRS}(1^\lambda), \mathsf{urs} \leftarrow \mathcal{URS})$, given $(x, \pi) \leftarrow \mathcal{A}^{\mathsf{H}}(\overline{\mathsf{crs}}; \rho)$, with probability at least $\frac{\mu(\lambda)^c - \mathrm{negl}(\lambda)}{p_{\mathsf{P}}(\lambda, Q_H)}$ the extractor finds $w \leftarrow \mathsf{Ext}(\overline{\mathsf{crs}}, \mathsf{td}, x, \pi, \rho, \vec{h})$ where $(x, w) \in \widetilde{\mathsf{R}}$. Here, $\vec{h}$ contains the outputs of H, the probability is over the random tape ρ of $\mathcal{A}$, the random tape of SimCRS, and the random choices of H. Also, we require that the runtime of Ext is bounded by $p_{\mathsf{T}}(\lambda, Q_H) \cdot \mathsf{Time}(\mathcal{A})$.

We further define *partial online-extractability* for NIZKs over a relation with statements $x = (x_0, x_1)$ and witnesses $w = (w_0, w_1)$. A NIZK is partially online-extractable if there exists an algorithm $\mathsf{Ext} = (\mathsf{Ext}_1, \mathsf{Ext}_2)$ such that Ext_1 samples a partial statement x_0 uniformly at random along with a trapdoor td and for any PPT adversary that outputs pairs of partial statements $x_{1,i}$ and proofs π_i such that all $((x_0, x_{1,i}), \pi_i)$ verify with probability $\mu(\lambda)$, the extraction algorithm Ext_1 can use the trapdoor to extract partial witnesses $w_{1,i}$ for all statements such that there exist partial witnesses $w_{0,i}$ with probability $\frac{\mu(\lambda) - \mathrm{negl}(\lambda)}{p_{\mathsf{P}}(\lambda, Q_H)}$ where p_{P} is a polynomial and Q_H is the number of hash queries made by the adversary. Looking forward, we will set the first partial statement x_0 to be the public parameters of a commitment scheme and the extracted witness to be the committed values - where the non-extracted witness is the opening of the commitments.

We also define *(statistical) adaptive subversion soundness*. Note that this notion does not require an extractor for the witness and the srs can be maliciously set up by an adversary, which differs from the standard notion of adaptive soundness. An NIZK is *(statistically) adaptively subversion sound* for relation $\widetilde{\mathsf{R}}$ inducing a language $\mathscr{L}_{\widetilde{\mathsf{R}}}$ if no (possibly unbounded) adversary, given a urs and access to the RO H, can output a subverted srs, an instance x, and a proof π such that $\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs} := (\mathsf{srs}, \mathsf{urs}), x, \pi) = 1$ but $x \notin \mathscr{L}_{\widetilde{\mathsf{R}}}$.

Fiat-Shamir transformation. We recall the Fiat-Shamir transformation [8, 2] to turn a Σ-protocol $\Sigma = (\mathsf{Init}, \mathsf{Chall}, \mathsf{Resp}, \mathsf{Verify})$ that satisfies *correctness*, *high-min entropy*, *honest verifier zero-knowledge*, and k-*special soundness*, into a NIZK $FS[\Sigma] = (\mathsf{GenSRS}, \mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$ using a random oracle H that maps to the challenge space $\mathcal{CH}$ of Σ:

- $\mathsf{GenSRS}(1^\lambda)$: outputs the empty string ϵ as we do not require a common reference string and omit crs as an input for other below algorithms,
- $\mathsf{Prove}^{\mathsf{H}}(x, w)$: receives a statement x and a witness w, runs $(\Omega, \mathsf{st}) \leftarrow \mathsf{Init}(x, w)$, computes the challenge $\gamma \leftarrow \mathsf{H}(x, \Omega)$, then computes $\tau \leftarrow \mathsf{Resp}(\mathsf{st}, \gamma)$ and outputs $\pi = (\Omega, \gamma, \tau)$.
- $\mathsf{Verify}^{\mathsf{H}}(x, \pi)$: receives a statement x and a proof $\pi = (\Omega, \gamma, \tau)$, checks that and outputs $b \leftarrow \mathsf{Verify}(x, \Omega, \gamma, \tau) \wedge \gamma = \mathsf{H}(x, \Omega)$.

The resulted NIZK satisfies *correctness*, *adaptive knowledge soundness* and *zero-knowledge*.

2 Full Preliminaries

2.1 Notation

Let $\lambda \in \mathbb{N}$ be the security parameter. A probabilistic polynomial time (PPT) algorithm $\mathcal{A}$ runs in time polynomial in the (implicit) security parameter λ. We write $\mathsf{Time}(\mathcal{A})$ for the runtime of $\mathcal{A}$. A function $f(\lambda)$ is *negligible* in λ if it is $\mathcal{O}(\lambda^{-c})$ for every $c \in \mathbb{N}$. We write $f = \mathrm{negl}(\lambda)$ for short. Similarly, we write $f = \mathsf{poly}(\lambda)$ if $f(\lambda)$ is a polynomial with variable λ. If D is a probability distribution, $x \leftarrow D$ means that x is sampled from D and if S is a set, $x \leftarrow S$ means that x is sampled uniformly and independently at random from S. We also write $|S|$ for the cardinality of set S. Further, we write $D_0 \overset{c}{\approx} D_1$ for distributions D_0, D_1, if for all PPT adversaries $\mathcal{A}$, we

[3] We remark that the soundness relation $\widetilde{\mathsf{R}}$ can be different from the (correctness) relation R. If $\widetilde{\mathsf{R}}$ is not explicitly defined, we implicitly set $\widetilde{\mathsf{R}} = \mathsf{R}$.

have $|\Pr[x_0 \leftarrow D_0 : \mathcal{A}(1^\lambda, x_0) = 1] - \Pr[x_1 \leftarrow D_1 : \mathcal{A}(1^\lambda, x_1) = 1]| = \mathsf{negl}(\lambda)$. Similarly, we write $D_0 \overset{s}{\approx} D_1$ if the above holds even for unbounded adversaries. For some PPT algorithm $\mathcal{A}$, we write $\mathcal{A}^{\mathcal{O}}$ if $\mathcal{A}$ has oracle access to the oracle $\mathcal{O}$. If $\mathcal{A}$ performs some check, and the check fails, we assume that $\mathcal{A}$ outputs $\perp$ immediately. Generally, we assume that adversaries are implicitly stateful.

We denote with $[n]$ the set $\{1, \ldots, n\}$ for $n \in \mathbb{N}$. We write $\mathbb{P}$ for the set of primes and $\mathbb{P}_I$ for the set of primes in the interval I. For some odd prime p, we use the representatives $\{-\frac{p-1}{2}, \cdots, \frac{p-1}{2}\}$ for $\mathbb{Z}_p$. For a group $\mathbb{G}$ we write $\mathsf{ord}(\mathbb{G})$ to denote the order of $\mathbb{G}$ and unless stated otherwise we write $\mathbb{G}$ with additive notation. We denote by $\mathsf{QR}_N = \{a \in \mathbb{Z}_N^* : \exists\, b \in \mathbb{Z}_N^*, b^2 \equiv a \mod N\}$ the quadratic residues $\mod N$. For some $N \in \mathbb{N}$, the group QR_N is a cyclic subgroup of $\mathbb{Z}_N^*$ and we denote by $\mathsf{Gen}(\mathsf{QR}_N)$ the set of generators of QR_N. We recall some properties of QR_N

Lemma 1 (Proposition 1, [6]). *Let $\lambda \in \mathbb{N}$ and $(N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda)$. Considering QR_N, the following holds:*

- *The group QR_N is cyclic of order $P'Q'$ where $P = 2P' + 1$ and $Q = 2Q' + 1$.*
- *$-1 \notin \mathsf{QR}_N$.*
- *Any square $h \in \mathsf{QR}_N$ has excatly four roots, among which there is exactly one square.*
- *For any element $h \in \mathsf{QR}_N$, finding roots of h is equivalent to factoring N.*
- *For $g, h \leftarrow \mathsf{QR}_N$, finding $a, b \in \mathbb{N} \setminus \{0\}$ such that $g^a \equiv h^b \mod N$ is equivalent to factoring N.*
- *For any $e \in \mathbb{N}$ coprime with $\phi(N)$ and $y \in \mathbb{Z}_N^*$, finding $x, e' \in \mathbb{N}$ such that $x^e \equiv y^{e'} \mod N$ is equivalent to finding an e-th root of y in $\mathbb{Z}_N^*$.*

2.2 Probability

Rejection Sampling. Let $V, L \in \mathbb{N}$. We define uniform rejection sampling for the interval $[0, V]$ with masking overhead L as in [5]. Let $v \in [0, V]$. To mask v additively with a mask μ via rejection sampling, perform the following steps.

1. Draw a random mask $\mu \leftarrow [0, (V+1)L]$.
2. Abort if $v + \mu \notin [V, (V+1)L]$.
3. Output $w = v + \mu$.

The value w is uniform over $[V, (V+1)L]$ conditioned on no abort and the abort probability is at most $1/L$. This is easy to see as it is a requirement for the abort that either

Noise Flooding. Let $V, L \in \mathbb{N}$. We define noise flooding for the interval $[-V, V]$ with masking overhead $L = 2^\lambda$. Let $v \in [-V, V]$. To mask v additively with a mask μ via noise flooding, output $w = v + \mu$, where $\mu \leftarrow [0, VL]$ is sampled at random. The value w is distributed close to uniform over $[0, VL]$ with statistical distance at most $1/L = \mathsf{negl}(\lambda)$.

Forking Lemma. We state here a version of Forking Lemma [14, 4] that fits our usage of it.

Lemma 2 (Lemma 1, [1]). *Let H be a set and let $F : H^q \to [q]$ be a possibly random function. For every $\vec{h} \in H^q$, let $E(\vec{h})$ be a probability event. The probability that when sampling k vectors $\vec{h}_1, \ldots, \vec{h}_k$ uniformly and independently at random (conditioned that vectors are identical on their first $F(\vec{h}_1)$ components), $E(\vec{h}_i)$ happens for all $i \in [k]$ and $F(\vec{h}_1) = F(\vec{h}_2) = \cdots = F(\vec{h}_k)$, is at least $\delta(E)^k/q^{k-1}$, where $\delta(E) \coloneqq \Pr[\vec{h} \leftarrow H^q : E(\vec{h})]$.*

2.3 Assumptions

Groups and RSA. Let GenG be a PPT algorithm that on input 1^λ and prime order p, outputs (a description of) a group $\mathbb{G} \leftarrow \mathsf{GenG}(1^\lambda)$ of order p. We generally use additive notations for prime order groups and capital letters for elements. Also, we assume that given the description, group operations and membership tests are efficient. We write $g \leftarrow \mathbb{G}$ for drawing elements from some group $\mathbb{G}$ at random. In the following, we assume that prime order groups are setup with GenG implicitly.

Let GenRSA be a PPT algorithm that on input 1^λ outputs $(N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda)$ such that $N = P \cdot Q$ with $P, Q \in \mathbb{P}$, where $P = 2P' + 1$ and $Q = 2Q' + 1$ are strong primes (*i.e.*, P', Q' are also primes). We assume that $P', Q' > 2^{\lambda+1}$.

Before recalling some standard hardness assumptions, let us recall the following well-known lemma.

Lemma 3. *Given $x, y \in \mathbb{Z}_N^*$ with $a, b \in \mathbb{Z}$ such that $x^a = y^b$ and $\gcd(a, b) = 1$, one can efficiently compute $\bar{x} \in \mathbb{Z}_N^*$ such that $\bar{x}^a = y$.*

Remark 2. We need the following well-known fact. Let $\mathbb{G}$ be a group and let $G \leftarrow \mathbb{G}$ be a random element from $\mathbb{G}$. Let $S \in \mathbb{N}$. We consider the problem of distinguishing zG, where $z \leftarrow [0, S]$, from $\tilde{z}G$ where $\tilde{z} \leftarrow \mathbb{Z}_{\mathsf{ord}(G)}$.

If the order p of the group $\mathbb{G}$ is known, then the distinguishing probability is 0 for $S = p - 1$. If only an upper bound U on the order is known, then the distinguishing probability is upper bounded by $1/L$ for $S = L \cdot U$. For the latter, we set $L = 2^\lambda$ throughout to obtain negligible distinguishing probability.

Next, we recall the definition of a relaxed DLOG-relation from [5] (for the hidden order group QR_N).

Definition 5 ((D, ℓ)-relaxed DLOG-relation). *Let $(N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda)$, $D, \ell \in \mathbb{N}$, and $\vec{g} = (g_0, \ldots, g_\ell) \in \mathsf{QR}_N^{\ell+1}$. Define the (D, ℓ)-*relaxed DLOG relation with regards to $\vec{g}$ *as*

$$\mathsf{R}_{D,\ell}(\vec{g}) = \left\{ (c, d, \{x_i\}_{i=1}^\ell) \;\middle|\; \begin{array}{c} c^d = \prod_{i=0}^\ell g_i^{x_i} \;\wedge\; \exists i \colon \frac{x_i}{d} \notin \mathbb{Z} \\ \wedge\, d \in [0, D] \;\wedge\; x_i \in \mathbb{Z} \end{array} \right\}$$

We define the advantage of $\mathcal{A}$ against the hardness of the (D, ℓ)-relaxed DLOG-relation as

$$\mathsf{Adv}^{\text{rel-dlog}}_{(D,\ell),\mathcal{A}}(\lambda) \coloneqq \Pr\left[\begin{array}{c} (N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda); g_0, \cdots, g_\ell \leftarrow \mathsf{Gen}(\mathsf{QR}_N); \\ (c, d, x_0, \ldots, x_\ell) \leftarrow \mathcal{A}(N, g_0, \ldots, g_\ell) \colon \\ (c, d, x_0, \ldots, x_\ell) \in \mathsf{R}_{D,\ell}(\vec{g}) \end{array} \right].$$

The following lemma is a simplification of Lemma A.13 of [5] sufficient for our purpose. Note that $\mathsf{ord}(\mathsf{QR}_N) = P'Q'$ and we assume that $P', Q' > 2^{\lambda+1}$.

Lemma 4. *Let $D \leq 2^{\lambda+1}$ and $\ell = \mathsf{poly}(\lambda)$. For every PPT adversary $\mathcal{A}$ we have that $\mathsf{Adv}^{\text{rel-dlog}}_{(D,\ell),\mathcal{A}}(\lambda) = \mathrm{negl}(\lambda)$ under the strong RSA assumption.*

Definition 6 (Decisional Diffie-Hellman). *In a cyclic group $\mathbb{G}$ of prime order p, which are set up w.r.t a security parameter $\lambda \in \mathbb{N}$, the* Decisional Diffie-Hellman (DDH) *assumption in $\mathbb{G}$ holds if for all PPT adversary $\mathcal{A}$ the advantage*

$$\begin{aligned} &\left|\Pr\left[G \leftarrow \mathbb{G}; a, b \leftarrow \mathbb{Z}_p \colon \mathcal{A}(G, aG, bG, abG) = 1\right]\right. \\ &\left. - \Pr\left[G \leftarrow \mathbb{G}; a, b, c \leftarrow \mathbb{Z}_p \colon \mathcal{A}(G, aG, bG, cG) = 1\right]\right| \end{aligned}$$

is negligible in λ.

Definition 7 (Strong RSA). *Let $\lambda \in \mathbb{N}$. The* strong RSA (sRSA) *assumption holds if for all PPT $\mathcal{A}$ the advantage*

$$\mathsf{Adv}^{\text{s-rsa}}_{\mathcal{A}}(\lambda) \coloneqq \Pr\left[\begin{array}{c} (N, P, Q) \leftarrow \mathsf{GenRSA}(1^\lambda); y \leftarrow \mathbb{Z}_N^* \\ (e, z) \leftarrow \mathcal{A}(N, y) \colon z^e \equiv y \bmod N \end{array} \right]$$

is negligible in λ.

2.4 Explaining Random Group Elements as Random Strings

For our framework, we require commitments with uniform public parameters pp. For readability, we allow pp (and also uniform random strings urs of NIZKs) to contain (uniform) group elements g of prime-order groups $\mathbb{G}$ with known order p. This is without loss of generality because with *explainable sampling*, we can explain $g \leftarrow \mathbb{G}$ as a random bitstring.

2.5 Commitment Scheme

Definition 8 (Commitment Scheme). *A* commitment scheme *is a tuple of algorithms* $\mathsf{C} = (\mathsf{C.Commit}, \mathsf{C.Verify})$ *such that*

- $\mathsf{C.Setup}(1^\lambda)$*: generates the public parameters* pp*,*
- $\mathsf{C.Commit}(\mathsf{pp}, m)$*: given the public parameters* pp*, message* $m \in \mathcal{C}_{\mathsf{msg}}$*, computes a commitment* $c \in \mathcal{C}_{\mathsf{com}}$ *with opening randomness* d*, and outputs the pair* (c, d)*,*
- $\mathsf{Verify}(\mathsf{pp}, c, m, d)$*: given the public parameters* pp*, message* $m \in \mathcal{C}_{\mathsf{msg}}$*, and opening randomness* d*, outputs a bit* $b \in \{0,1\}$ *which depends on the validity of the opening* (m, d) *with respect to the commitment* c*.*

Here, $\mathcal{C}_{\mathsf{msg}}$*,* $\mathcal{C}_{\mathsf{rnd}}$*,* $\mathcal{C}_{\mathsf{com}}$*, are message, randomness, and commitment spaces, respectively. If the public parameters are uniform or explainable* as per *Section 2.4 (*i.e.*,* Setup *outputs some* $\mathsf{pp} \leftarrow \{0,1\}^\ell$ *for* $\ell \in \mathbb{N}$*) we omit* Setup *without loss of generality.*

Below, we define the correctness, hiding and binding properties of a commitment scheme.

Definition 9 (Correctness). *A commitment scheme is* correct*, if for all* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), m \in \mathcal{C}_{\mathsf{msg}}, r \in \mathcal{C}_{\mathsf{rnd}}, (c,d) \leftarrow \mathsf{Commit}(\mathsf{pp}, m; r)$*, it holds that* $\mathsf{Verify}(\mathsf{pp}, c, m, d) = 1$*.*

Definition 10 (Hiding). *A commitment scheme is* hiding *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A}}(\lambda) = \left| \Pr \begin{bmatrix} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), (m_0, m_1) \leftarrow \mathcal{A}(\mathsf{pp}), \\ m_0, m_1 \in \mathcal{C}_{\mathsf{msg}}, \mathsf{coin} \leftarrow \{0,1\} \\ (c,d) \leftarrow \mathsf{Commit}(\mathsf{pp}, m_{\mathsf{coin}}), \end{bmatrix} : \mathsf{coin} = \mathcal{A}(c) \right] - \frac{1}{2} \right| = \mathrm{negl}(\lambda).$$

Definition 11 (Binding). *A commitment scheme is* binding *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\lambda) = \Pr \begin{bmatrix} \mathsf{pp} \leftarrow \{0,1\}^{\ell_{\mathsf{C}}}, \\ (c, m_0, m_1, d_0, d_1) \leftarrow \mathcal{A}(\mathsf{pp}) \end{bmatrix} : \begin{matrix} m_0 \neq m_1 \in \mathcal{C}_{\mathsf{msg}} \\ \mathsf{Verify}(\mathsf{pp}, c, m_b, d_b) = 1, b \in \{0,1\} \end{matrix} \Bigg] = \mathrm{negl}(\lambda).$$

Remark 3. A commitment scheme is said to be *perfectly binding* if for any (possibly unbounded) $\mathcal{A}$, it holds that $\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\lambda) = 0$.

(Bounded) Integer Commitments. We refer to a commitment scheme with message space $[\mathbb{A}, B] \subseteq \mathbb{N}$ as a (bounded) integer commitment scheme. We often omit the term bounded if the message space is clear by context.

ElGamal commitments. We recall ElGamal (EG) over a group $\mathbb{G}$ of prime order p with message space $\mathbb{Z}_p$ [7]. We use additive notation for prime order groups.

- $\mathsf{EG.GenPP}(1^\lambda)$: set $(G, H) \leftarrow \mathbb{G} \setminus \{0\}$ and output $\mathsf{pp} = (G, H)$.
- $\mathsf{EG.Commit}(\mathsf{pp}, m)$: sample $r \leftarrow \mathbb{Z}_p$ and set $c = (mG + rH, rG)$, and output (c, r).
- $\mathsf{EG.Verify}(\mathsf{pp}, c, m, r)$: check if $c = (mG + rH, rG)$.

Note that the public parameters are uniform and we can sample them via a random oracle to avoid trusted setup. EG commitments are correct, hiding under DDH and perfectly binding.

Remark 4. If in verification of EG, we check that $m \in [0, M]$ for $M < p$, then m is fixed over the integers and we can interpret the commitment as an integer commitment with message space $[0, M] \subseteq \mathbb{N}$.

Pedersen Commitments in $\mathbf{QR}_N$ We recall Pedersen multi-commitments (MPed) over QR_N with message space $\mathbb{Z}^\ell$ for some $\ell \in \mathbb{N}$ [6].

- $\mathsf{MPed.GenPP}(1^\lambda)$: set $(N, P, Q) \leftarrow \mathsf{Gen}(1^\lambda)$ and sample ℓ random generators g_i of QR_N, and output $\mathsf{pp} = (N, h, g_1, \cdots, g_\ell)$. Note that with (P, Q), we can check whether g_i generates QR_N.
- $\mathsf{MPed.Commit}(\mathsf{pp}, \vec{m})$: sample $r \leftarrow [0, N \cdot 2^\lambda]$, set $c \leftarrow h^r \cdot \prod_{i=1}^{\ell} g_i^{m_i} \bmod N$, and output (c, r).
- $\mathsf{MPed.Verify}(\mathsf{pp}, c, \vec{m}, r)$: check if $c = \pm h^r \cdot \prod_{i=1}^{\ell} g_i^{m_i} \bmod N$.

MPed commitments are correct, statistically hiding and binding under the factoring assumption (which is implied by sRSA). Throughout this work, we use MPed commitments in QR_N to enforce in security proofs that values extracted from NIZKs are integers via lemma 4.

2.6 Signature Scheme

Definition 12 (Signature Scheme). *A* signature scheme *is a tuple of PPT algorithms* $\mathsf{S} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *such that*

- $\mathsf{KeyGen}(1^\lambda)$*: generates a verification key* vk *and a signing key* sk*,*
- $\mathsf{Sign}(\mathsf{sk}, m)$*: given a signing key* sk *and a message* $m \in \mathcal{S}_{\mathsf{msg}}$*, outputs a signature* σ*,*
- $\mathsf{Verify}(\mathsf{vk}, m, \sigma)$*: given a verification key* pk *and a signature* σ *on message* m*,* deterministically *outputs a bit* $b \in \{0,1\}$*.*

Here, $\mathcal{S}_{\mathsf{msg}}$ *is the message space.*

We define the standard notion of correctness and `euf-cma` security

Definition 13 (Correctness). *A signature scheme is* correct*, if for all* $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$*,* $m \in \mathcal{S}_{\mathsf{msg}}$*, and* $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$*, it holds that* $\mathsf{Verify}(\mathsf{vk}, m, \sigma) = 1$*.*

Definition 14 (EUF-CMA). *A signature scheme is* `euf-cma` *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{euf}}_{\mathcal{A}}(\lambda) = \Pr\left[\begin{matrix}(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda) \\ (m, \sigma) \leftarrow \mathcal{A}^{\mathsf{Sign}(\mathsf{sk}, \cdot)}(\mathsf{vk})\end{matrix} \quad : m \notin L \wedge \ \mathsf{Verify}(\mathsf{vk}, m, \sigma) = 1\right] = \mathrm{negl}(\lambda),$$

where L *is the list of messages* $\mathcal{A}$ *queried to the* Sign*-oracle.*

2.7 Blind Signatures

Definition 15 (Blind Signature).

- $\mathsf{BSGen}(1^\lambda)$*: returns the verification key* bvk *and signing key* bsk*,*
- $\mathsf{BSUser}(\mathsf{bvk}, m)$*: given verification key* pk *and message* m*, outputs a first message* ρ_1 *and a state* st*,*
- $\mathsf{BSSigner}(\mathsf{bsk}, \rho_1)$*: given signing key* sk *and first message* ρ_1*, outputs a second message* ρ_2*,*
- $\mathsf{BSDerive}(\mathsf{st}, \rho_2)$*: given state* st *and second message* ρ_2*, outputs a signature* σ*,*
- $\mathsf{BSVerify}(\mathsf{bvk}, m, \sigma)$*: given verification key* bvk *and signature* σ *on message* m*, returns a bit* $b \in \{0,1\}$*.*

The state is kept implicit in the following.

Definition 16 (Correctness). *A blind signature scheme is correct, if for all messages* m*,* $(\mathsf{bvk}, \mathsf{bsk}) \leftarrow \mathsf{BSGen}(1^\lambda), (\rho_1, \mathsf{st}) \leftarrow \mathsf{BSUser}(\mathsf{bvk}, m), \rho_2 \leftarrow \mathsf{BSSigner}(\mathsf{bsk}, \rho_1), \sigma \leftarrow \mathsf{BSDerive}(\mathsf{st}, \rho_2)$*, it holds that* $\mathsf{BSVerify}(\mathsf{bvk}, m, \sigma) = 1$ *with probability at least* $1 - \mathrm{negl}(\lambda)$*.*

Definition 17 (Blindness). *A blind signature scheme is* blind under malicious keys *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{blind}}_{\mathcal{A}}(\lambda) = \left| \Pr\left[\begin{matrix}(\mathsf{bvk}, m_0, m_1) \leftarrow \mathcal{A}(1^\lambda),\ \mathsf{coin} \leftarrow \{0,1\}, \\ (\rho_{1,b}, \mathsf{st}_b) \leftarrow \mathsf{BSUser}(\mathsf{bvk}, m_b) \text{ for } b \in \{0,1\}, \\ (\rho_{2,\mathsf{coin}}, \rho_{2,1-\mathsf{coin}}) \leftarrow \mathcal{A}(\rho_{1,\mathsf{coin}}, \rho_{1-\mathsf{coin}}), \\ \sigma_b \leftarrow \mathsf{BSDerive}(\mathsf{st}_b, \rho_{2,b}) \text{ for } b \in \{0,1\}, \\ \text{if } \exists b \text{ s.t. } \mathsf{BSVerify}(\mathsf{bvk}, m_b, \sigma_b) = 0\text{:} \\ \quad \text{then } \sigma_0 = \sigma_1 = \bot,\end{matrix} \quad : \mathsf{coin} = \mathcal{A}(\sigma_0, \sigma_1)\right] - \frac{1}{2}\right| \leq \mathrm{negl}(\lambda).$$

Definition 18 (One-more Unforgeability). *A blind signature scheme is* one-more unforgeable *if for any* $Q = \mathsf{poly}(\lambda)$ *and PPT adversary* $\mathcal{A}$ *that makes at most* Q *signing queries, we have*

$$\mathsf{Adv}^{\mathrm{omuf}}_{\mathcal{A}}(\lambda) = \Pr\left[\begin{matrix}(\mathsf{bvk}, \mathsf{bsk}) \leftarrow \mathsf{BSGen}(1^\lambda) \\ \{(m_i, \sigma_i)\}_{i=1}^{Q+1} \\ \mathcal{A}^{\mathsf{BSSigner}(\mathsf{bsk}, \cdot)}(\mathsf{bvk})\end{matrix} \quad \leftarrow : \begin{matrix}\forall i \neq j \in [Q+1] : m_i \neq m_j \\ \wedge\ \mathsf{BSVerify}(\mathsf{bvk}, m_i, \sigma_i) = 1\end{matrix}\right] = \mathrm{negl}(\lambda).$$

2.8 Σ-Protocol

Let R be an NP relation with statements x and witnesses w. We denote by $\mathscr{L}_{\mathsf{R}} = \{x \mid \exists w \text{ s.t. } (x,w) \in \mathsf{R}\}$ the language induced by R. A Σ-protocol for an NP relation R for language $\mathscr{L}_{\mathsf{R}}$ is a tuple of PPT algorithms $\Sigma = (\mathsf{Init}, \mathsf{Chall}, \mathsf{Resp}, \mathsf{Verify})$ such that

- $\mathsf{Init}(x,w)$: given a statement $x \in \mathscr{L}_{\mathsf{R}}$, and a witness w such that $(x,w) \in \mathsf{R}$, outputs a first flow message (*i.e.*, commitment) Ω and a state st, where we assume st includes x, w,
- $\mathsf{Chall}()$: samples a challenge $\gamma \leftarrow \mathcal{CH}$ (without taking any input),
- $\mathsf{Resp}(\mathsf{st}, \gamma)$: given a state st and a challenge $\gamma \in \mathcal{CH}$, outputs a third flow message (*i.e.*, response) τ,
- $\mathsf{Verify}(x, \Omega, \gamma, \tau)$: given a statement $x \in \mathscr{L}_{\mathsf{R}}$, a commitment Ω, a challenge $\gamma \in \mathcal{CH}$, and a response τ, outputs a bit $b \in \{0,1\}$.

Definition 19 (Correctness). *A Σ-protocol is* correct*, if for all $(x,w) \in \mathsf{R}$, $(\Omega, \mathsf{st}) \leftarrow \mathsf{Init}(x,w)$, $\gamma \in \mathcal{CH}$, and $\tau \leftarrow \mathsf{Resp}(\mathsf{st}, \gamma)$, it holds that $\mathsf{Verify}(x, \Omega, \gamma, \tau) = 1$.*

Definition 20 (High Min-Entropy). *A Σ-protocol has* high min-entropy *if for all $(x,w) \in \mathsf{R}$ and (possibly unbounded) adversary $\mathcal{A}$, it holds that*

$$\Pr[(\Omega, \mathsf{st}) \leftarrow \mathsf{Init}(x,w), \Omega' \leftarrow \mathcal{A}(1^\lambda) : \Omega = \Omega'] = \mathrm{negl}(\lambda).$$

Definition 21 (Non-abort HVZK). *A Σ-protocol is* non-abort honest-verifier zero-knowledge *(HVZK), if there exists a PPT zero-knowledge simulator Sim such that the distributions of $\mathsf{Sim}(x,\gamma)$ and the honestly generated transcript with Init initialized with (x,w) are statistically indistinguishable for any $x \in \mathscr{L}_{\mathsf{R}}$, and $\gamma \in \mathcal{CH}$, where the honest execution is conditioned on γ being used as the challenge and no abort occurring.*

We write HVZK for short if the Σ-protocol never aborts.

Definition 22 (k-Special Soundness). *A Σ-protocol is k-*special sound*, if there exists a* deterministic *PT extractor Ext such that given k valid transcripts $\{(\Omega, \gamma_i, \tau_i)\}_{i \in [k]}$ for statement x with pairwise distinct challenges $(\gamma_i)_i$, outputs a witness w such that $(x,w) \in \mathsf{R}$.*

2.9 Non-Interactive Zero Knowledge

Let $\mathcal{URS} = \{0,1\}^\ell$ be a set of *uniform random strings* for some $\ell \in \mathbb{N}$ and $\mathcal{SRS}$ be some set of *structured random strings* with efficient membership test [4]. An NIZK for a relation R with common reference string space $\mathcal{CRS} = \mathcal{SRS} \times \mathcal{URS}$ is a tuple of PPT algorithms $(\mathsf{GenSRS}, \mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$, where the latter two are oracle-calling, such that:

- $\mathsf{GenSRS}(1^\lambda)$: outputs a structured reference string $\mathsf{srs} \in \mathcal{SRS}$,
- $\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, x, w)$: receives a $\mathsf{crs} = (\mathsf{srs}, \mathsf{urs}) \in \mathcal{CRS}$, a statement x and a witness w, and outputs a proof π,
- $\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, x, \pi)$: receives a $\mathsf{crs} = (\mathsf{srs}, \mathsf{urs}) \in \mathcal{CRS}$, a statement x and a proof π, and outputs a bit $b \in \{0,1\}$.

We recall that $\mathscr{L}_{\mathsf{R}} = \{x \mid \exists w \ : \ (x,w) \in \mathsf{R}\}$ denotes the language induced by R. If there is no crs needed, *i.e.* $\mathcal{CRS} = \varnothing$, we then omit crs as an input to Prove and Verify.

Definition 23 (Correctness). *An NIZK is* correct *if for any $\mathsf{crs} = (\mathsf{srs}, \mathsf{urs})$ with $\mathsf{srs} \leftarrow \mathsf{GenSRS}(1^\lambda)$ and $\mathsf{urs} \leftarrow \mathcal{URS}$, $(x,w) \in \mathsf{R}$, and $\pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, x, w)$, it holds that $\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, x, \pi) = 1$.*

[4] This membership test is required for our definition of subversion zero-knowledge. Note that in general it is difficult to check that some srs was generated via GenSRS. (We allow that $\mathcal{SRS}$ is not equal to the output space of GenSRS.)

Definition 24 (Zero-Knowledge). *An* NIZK *is* zero-knowledge *(*ZK*) if there exists a PPT simulator* $\mathsf{Sim} = (\mathsf{Sim}_{\mathsf{crs}}, \mathsf{Sim}_{\mathsf{H}}, \mathsf{Sim}_{\pi})$ *such that for any PPT adversary* $\mathcal{A}$*, it holds that*

$$\mathsf{Adv}^{\mathrm{zk}}_{\mathcal{A}}(\lambda) = \left| \Pr\left[\begin{matrix}\mathsf{srs} \leftarrow \mathsf{GenSRS}(1^\lambda),\\ \mathsf{crs} = (\mathsf{srs}, \mathsf{urs}),\\ \mathcal{A}^{\mathsf{H},\mathcal{P}}(\mathsf{crs}) = 1\end{matrix}\right] - \Pr\left[\begin{matrix}\mathsf{crs} \leftarrow \mathsf{Sim}_{\mathsf{crs}}(1^\lambda),\\ \mathsf{crs} = (\mathsf{srs}, \mathsf{urs}),\\ \mathcal{A}^{\mathsf{Sim}_{\mathsf{H}},\mathcal{S}}(\mathsf{crs}) = 1\end{matrix}\right] \right| = \mathrm{negl}(\lambda),$$

where $\mathcal{P}$ *and* $\mathcal{S}$ *are oracles that on input* (x, w) *return* $\perp$ *if* $(x, w) \notin \mathsf{R}$*, and else output* $\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, x, w)$ *or* $\mathsf{Sim}_{\pi}(\mathsf{crs}, x)$ *respectively. Note that the probability is taken over the randomness of* Sim *and* $\mathcal{A}$*, and the random choices of* H *and* urs*. Also,* $\mathsf{Sim}_{\mathsf{crs}}, \mathsf{Sim}_{\mathsf{H}}$ *and* Sim_{π} *have a shared state.*

We also define a notion of *subversion zero-knowledge*, inspired by the notion introduced in [3]. Informally, it guarantees that zero-knowledge holds even for a malicious crs.

Definition 25 (Subversion Zero-Knowledge). *An* NIZK *is* subversion zero-knowledge *(*Sub-ZK*) if there exists a PPT simulator* $\mathsf{Sim} = (\mathsf{Sim}_{\mathsf{H}}, \mathsf{Sim}_{\pi})$ *such that for any PPT adversary* $\mathcal{A}$*, it holds that*

$$\mathsf{Adv}^{\text{sub-zk}}_{\mathcal{A}}(\lambda) = \left| \Pr\left[\begin{matrix}\mathsf{urs} \leftarrow \mathcal{URS},\\ (\mathsf{srs}, \mathsf{st}) \leftarrow \mathcal{A}^{\mathsf{H}}(\mathsf{urs}),\\ \mathsf{crs} = (\mathsf{srs}, \mathsf{urs}),\\ \mathcal{A}^{\mathsf{H},\mathcal{P}}(\mathsf{st}) = 1 \wedge \mathsf{srs} \in \mathcal{SRS}\end{matrix}\right] - \Pr\left[\begin{matrix}\mathsf{urs} \leftarrow \mathcal{URS},\\ (\mathsf{srs}, \mathsf{st}) \leftarrow \mathcal{A}^{\mathsf{Sim}_{\mathsf{H}}}(\mathsf{urs}),\\ \mathsf{crs} = (\mathsf{srs}, \mathsf{urs}),\\ \mathcal{A}^{\mathsf{Sim}_{\mathsf{H}},\mathcal{S}}(\mathsf{st}) = 1 \wedge \mathsf{srs} \in \mathcal{SRS}\end{matrix}\right] \right| = \mathrm{negl}(\lambda),$$

where $\mathcal{P}$ *and* $\mathcal{S}$ *are oracles that on input* (x, w) *return* $\perp$ *if* $(x, w) \notin \mathsf{R}$*, and else output* $\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, x, w)$ *or* $\mathsf{Sim}_{\pi}(\mathsf{crs}, x)$*, respectively. Note that the probability is taken over the randomness of* Sim *and* $\mathcal{A}$*, and the random choices of* H *and* urs*. Also, both* $\mathsf{Sim}_{\mathsf{H}}$ *and* Sim_{π} *have a shared state.*

We define different notions of soundness. We remark that the soundness relation $\widetilde{\mathsf{R}}$ can be different from the (correctness) relation R. If $\widetilde{\mathsf{R}}$ is not explicitly defined, we implicitly set $\widetilde{\mathsf{R}} = \mathsf{R}$.

Definition 26 (Adaptive Knowledge Soundness). *An* NIZK *is* adaptively knowledge sound *for relation* $\widetilde{\mathsf{R}}$ *if there exists PPT simulator* SimCRS *and extractor* Ext *such that*

CRS Indistinguishability. *For any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathrm{crs}}_{\mathcal{A}}(\lambda) = \left| \Pr\left[\begin{matrix}\mathsf{srs} \leftarrow \mathsf{GenSRS}(1^\lambda), \mathsf{urs} \leftarrow \mathcal{URS},\\ \mathsf{crs} = (\mathsf{srs}, \mathsf{urs}) : \mathcal{A}^{\mathsf{H}}(\mathsf{crs}) = 1\end{matrix}\right] - \Pr\left[\begin{matrix}(\overline{\mathsf{crs}}, \mathsf{td}) \leftarrow \mathsf{SimCRS}(1^\lambda) :\\ \mathcal{A}^{\mathsf{H}}(\overline{\mathsf{crs}}) = 1\end{matrix}\right] \right| = \mathrm{negl}(\lambda),$$

Knowledge Soundness. *There exists positive polynomials* $p_{\mathsf{T}}, p_{\mathsf{P}}$ *and a constant* c *such that given oracle access to any PPT adversary* $\mathcal{A}$ *(with explicit random tape* ρ*) that makes* $Q_H = \mathsf{poly}(\lambda)$ *random oracle queries with*

$$\Pr[(\overline{\mathsf{crs}}, \mathsf{td}) \leftarrow \mathsf{SimCRS}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}^{\mathsf{H}}(\overline{\mathsf{crs}}; \rho) : \mathsf{Verify}^{\mathsf{H}}(\overline{\mathsf{crs}}, x, \pi) = 1] \geq \mu(\lambda),$$

we have

$$\Pr\left[\begin{matrix}(\overline{\mathsf{crs}}, \mathsf{td}) \leftarrow \mathsf{SimCRS}(1^\lambda),\\ (x, \pi) \leftarrow \mathcal{A}^{\mathsf{H}}(\overline{\mathsf{crs}}; \rho),\\ w \leftarrow \mathsf{Ext}(\overline{\mathsf{crs}}, \mathsf{td}, x, \pi, \rho, \vec{h})\end{matrix} \quad : (x, w) \in \widetilde{\mathsf{R}} \right] \geq \frac{\mu(\lambda)^c - \mathrm{negl}(\lambda)}{p_{\mathsf{P}}(\lambda, Q_H)},$$

where $\vec{h}$ *are the outputs of* H*, and the probability is over the random tape* ρ *of* $\mathcal{A}$*, the random tape of* SimCRS*, and the random choices of* H*. Also, we require that the runtime of* Ext *is bounded by* $p_{\mathsf{T}}(\lambda, Q_H) \cdot \mathsf{Time}(\mathcal{A})$*.*

We also adapt the standard notion of online-extractability in two ways. Instead of embedding the online-extraction trapdoor td into crs, we allow that the extractor embeds it into specific parts of statement. Also, we relax the requirements in the sense that only a partial witness w_1 is extracted. For extraction, we require that there exists a witness w_0 such that $(x, (w_0, w_1)) \in \widetilde{\mathsf{R}}$.

Definition 27 (Partial Online Extractability). *An* NIZK *is* partially online-extractable *for relation* $\widetilde{\mathsf{R}}$ *with statements* $x = (x_0, x_1)$ *and witnesses* $w = (w_0, w_1)$*, where* $w_0 \in W_0$ *and* $x_0 \in X_0$ *for some sets* W_0, X_0*, if for all PPT adversaries* $\mathcal{A}$*, there exists a stateful PPT extractor* $\mathsf{Ext} = (\mathsf{Ext}_1, \mathsf{Ext}_2)$*, such that*

1. x_0 *is distributed uniform over* X_0 *for* $(x_0, \mathsf{td}) \leftarrow \mathsf{Ext}_1(1^\lambda)$ *and*
2. *there exists positive polynomials* $p_\mathsf{T}, p_\mathsf{P}$ *such that for any* $Q_H = \mathsf{poly}(\lambda)$ *and PPT adversary* $\mathcal{A}$ *that makes at most* Q_H *random oracle queries with*

$$\Pr\left[\begin{array}{c}(x_0, \mathsf{td}) \leftarrow \mathsf{Ext}_1(1^\lambda), \mathsf{crs} \leftarrow \mathsf{GenSRS}(1^\lambda),\\ \{(x_{1,i}, \pi_i)\}_{i\in[Q_S]} \leftarrow \mathcal{A}^{\mathsf{H}}(\mathsf{crs}, x_0), x_i \leftarrow (x_0, x_{1,i}) :\\ \forall i \in [Q_S] : \mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, x_i, \pi_i) = 1\end{array}\right] \geq \mu(\lambda),$$

it holds that

$$\Pr\left[\begin{array}{c}(x_0, \mathsf{td}) \leftarrow \mathsf{Ext}_1(1^\lambda), \mathsf{crs} \leftarrow \mathsf{GenSRS}(1^\lambda),\\ \{(x_{1,i}, \pi_i)\}_{i\in[Q_S]} \leftarrow \mathcal{A}^{\mathsf{H}}(\mathsf{crs}, x_0), x_i \leftarrow (x_0, x_{1,i})\\ \{w_{1,i} \leftarrow \mathsf{Ext}_2(\mathsf{crs}, \mathsf{td}, x_i, \pi_i)\}_{i\in[Q_S]} :\\ \forall i \in [Q_S]\ \exists w_{0,i} \in W_0 : (x_i, (w_{0,i}, w_{1,i})) \in \widetilde{\mathsf{R}}\\ \wedge\ \mathsf{Verify}^{\mathsf{H}}(\overline{\mathsf{crs}}, x_i, \pi_i) = 1\end{array}\right] \geq \frac{\mu(\lambda) - \mathrm{negl}(\lambda)}{p_\mathsf{P}(\lambda, Q_H)},$$

where the runtime of Ext *is upper bounded by* $p_\mathsf{T}(\lambda, Q_H) \cdot \mathsf{Time}(\mathcal{A})$*.*

Adaptive Subversion Soundness. We also define adaptive subversion soundness, where we allow that srs can be maliciously set up by an adversary. Note that this notion does not require an extractor for the witness.

Definition 28 (Statistical Adaptive Subversion Soundness). *An* NIZK *is* (statistically) adaptively sound *for relation* $\widetilde{\mathsf{R}}$ *inducing a language* $\mathscr{L}_{\widetilde{\mathsf{R}}}$ *if for any possibly unbounded* $\mathcal{A}$ *we have*

$$\mathsf{Adv}^{\mathrm{snd}}_{\mathcal{A}}(\lambda) \coloneqq \Pr\left[\begin{array}{ll}\mathsf{urs} \leftarrow \mathcal{URS}, & \\ (\mathsf{srs}, x, \pi) \leftarrow \mathcal{A}^{\mathsf{H}}(1^\lambda; \mathsf{urs}), & : \begin{array}{l} x \notin \mathscr{L}_{\widetilde{\mathsf{R}}},\\ \mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, x, \pi) = 1\end{array}\\ \mathsf{crs} \leftarrow (\mathsf{srs}, \mathsf{urs}) & \end{array}\right] \leq \mathrm{negl}(\lambda),$$

where the probability is over the random coins of $\mathcal{A}$ *and* GenCRS*, the random choices of* urs*, and the random choices of* H*.*

Fiat-Shamir transformation. We recall the Fiat-Shamir transformation [8] to turn a Σ-protocol into a NIZK. Sometimes, we require more involved variants of this transformations. In that case, we provide the compiled NIZK explicitly.

Theorem 1. *Let* $\Sigma = (\mathsf{Init}, \mathsf{Chall}, \mathsf{Resp}, \mathsf{Verify})$ *be a* Σ*-protocol that satisfies* correctness*,* high-min entropy*,* honest verifier zero-knowledge*, and* 2-Special Soundness*. The* Fiat-Shamir transformation $FS[\Sigma] = (\mathsf{GenSRS}, \mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$ *is described below:*

- $\mathsf{GenSRS}(1^\lambda)$*: outputs the empty string* ϵ *as we do not require a common reference string and omit* crs *as an input for other below algorithms,*
- $\mathsf{Prove}^{\mathsf{H}}(x, w)$*: receives a statement* x *and a witness* w*, runs* $(\Omega, \mathsf{st}) \leftarrow \mathsf{Init}(x, w)$*, computes the challenge* $\gamma \leftarrow \mathsf{H}(x, \Omega)$*, then computes* $\tau \leftarrow \mathsf{Resp}(\mathsf{st}, \gamma)$ *and outputs* $\pi = (\Omega, \gamma, \tau)$*.*
- $\mathsf{Verify}^{\mathsf{H}}(x, \pi)$*: receives a statement* x *and a proof* $\pi = (\Omega, \gamma, \tau)$*, and outputs* $b \leftarrow \mathsf{Verify}(x, \Omega, \gamma, \tau) \wedge \gamma = \mathsf{H}(x, \Omega)$*.*

In the ROM, $FS[\Sigma]$ *is a* NIZK *that is* correct *and satisfies* adaptive knowledge soundness*.*

3 Constructions from OTs

The construction $\mathsf{BS}_{\mathsf{fis}}$ is given in Figure 1. The public parameters are set to contain:

- A prime-order bilinear group setting $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathsf{t}, g_1, g_2, g_\mathsf{t}, \mathsf{e}, p)$ and $\mathbb{G}_1, \mathbb{G}_2$ are both written multiplicatively.
- A random $h_s \in \mathbb{G}_1$, wit independent $\vec{u} \coloneqq (u_i)_{i=0}^{\ell} \in \mathbb{G}_1^{\ell+1}$ for the Waters hash function $\mathcal{F}(\mathcal{M}) \coloneqq u_0 \cdot \prod_i u_i^{M_i}$ on messages $\mathcal{M} \coloneqq (M_i)_i$.
- A vector $\mathsf{ek} \coloneqq (h_i)_i \coloneqq (g_1^{x_i})_i$ and $g_s \coloneqq \prod_i h_i$, by choosing ℓ random scalars $(x_i)_{i=1}^{\ell} \in \mathbb{Z}_p^\ell$.

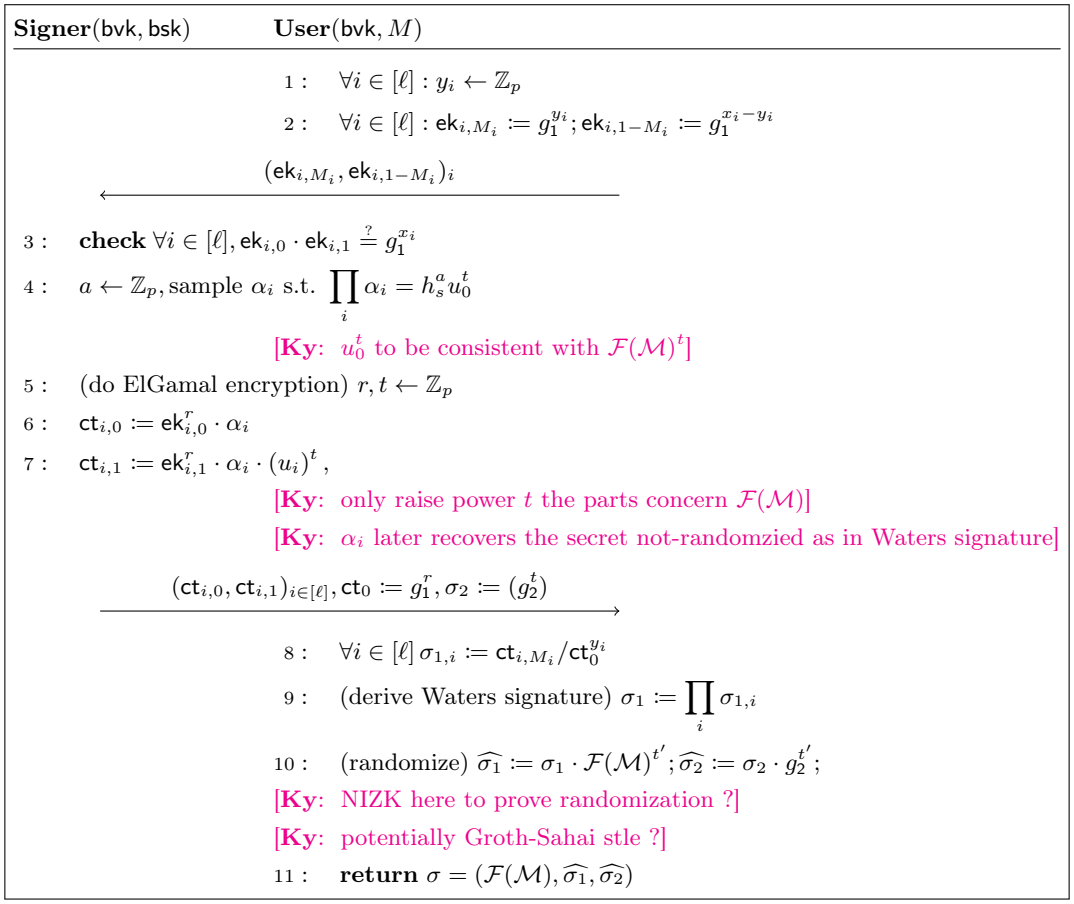

Fig. 1: A signing session of BS for message m. We have $\mathsf{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathsf{t}, g_1, g_2, g_\mathsf{t}, \mathsf{e}, p, \mathsf{ek}, g_s, h_s, \vec{u})$. The keys are $\mathsf{bvk} = (g_2^a), \mathsf{bsk} = h_s^a$. If a check fails, the party aborts.

Correctness. The derived $\sigma = (\sigma_1, \sigma_2)$ is a Waters signature. Verification using $\mathsf{bvk} = (g_2^a)$ by:

$$\mathbf{e}(h_s, \mathsf{bvk}) \cdot \mathbf{e}(\mathcal{F}(\mathcal{M}), \widehat{\sigma_2}) \stackrel{?}{=} \mathbf{e}(\widehat{\sigma_1}, g_2) \ .$$

Honnest-Verifier Blindness. stteps:

- ZK simulation for the randomization step, movinng dependence on $\mathcal{F}(\mathcal{M}), t'$ w.r.t $\widehat{\sigma_1}, \widehat{\sigma_2}$.
- simulate $\sigma_1 \coloneqq \widehat{\sigma_1}/\mathcal{F}(\vec{0})^{t_0}$ where $\mathcal{F}(\vec{0})$ and t_0 (comes from the ZK simulator) are independent from $\mathcal{M}$. Output $\mathcal{F}(\vec{0})$ in the derived signature.
- then smaple $\sigma_{1,i}$ so that $\prod_i \sigma_{1,i} = \sigma_1$. Now the derivation step does not rely on $\mathcal{M}$ anymore.
- Make the first flow indep from $\mathcal{M}$: sample $\mathsf{ek}_{i,0}, \mathsf{ek}_{i,1}$ such that their product is $g_1^{x_i}$ (part of public parametre.)

4 Constructions from Dual-Mode OTs

We change the ElGamal-based OT in Figure 1 by the *dual-mode* OT from [13]. This dual-mode will become handy later for the OMUF proof.

Definition 29 (co-CDH [15]). *For a bilinear group as above,*

$$\mathsf{Adv}_{\mathcal{B}}^{\mathsf{co\text{-}CDH}}(\lambda) \ := \ \Pr\big[\, \mathcal{B}(g_1, g_2, g_1^a, g_2^a, g_1^b) = g_1^{ab} \ : \ a, b \leftarrow \mathbb{Z}_p \,\big].$$

The co-CDH *assumption states that this is negligible for all PPT* $\mathcal{B}$.

4.1 The protocol

[**Ky**: To change to same figure-format] The change relative to the draft is confined to the OT layer: one global CRS 4-tuple, one extra element per first-flow position, two-element ciphertexts with fresh randomness, plus F1–F3.

BSGen. Bilinear group as above; $h_s \leftarrow \mathbb{G}_1$; Waters vector $u_0, \ldots, u_\ell \leftarrow \mathbb{G}_1$; OT-CRS $(g, h, w, \tilde{w}) \leftarrow \mathbb{G}_1^4$ *uniform.* Write $\eta := \log_g h$ when $g \neq 1$. Sample $a \leftarrow \mathbb{Z}_p$, set $\mathsf{bvk} = g_2^a$, $\mathsf{bsk} = h_s^a$. Output $\mathsf{pp} = (\text{group}, h_s, (u_i)_i, (g, h, w, \tilde{w}))$.

BSUser(bvk, M), $M \in \{0,1\}^\ell$. For $i \in [\ell]$: $y_i \leftarrow \mathbb{Z}_p$;

$$(\mathsf{ek}_{i,M_i}, \widetilde{\mathsf{ek}}_{i,M_i}) := (g^{y_i}, h^{y_i}), \qquad (\mathsf{ek}_{i,1-M_i}, \widetilde{\mathsf{ek}}_{i,1-M_i}) := \big(w \cdot \mathsf{ek}_{i,M_i}^{-1},\ \tilde{w} \cdot \widetilde{\mathsf{ek}}_{i,M_i}^{-1}\big).$$

$\rho_1 := \big(\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0}\big)_{i\in[\ell]}$; $\mathsf{st} := (M, (y_i)_i)$.

BSSigner(bsk, ρ_1). [**Ky**: $\rho_1 := \big(\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0}\big)_{i\in[\ell]}$, The signer derives $(\mathsf{ek}_{i,1}, \widetilde{\mathsf{ek}}_{i,1}) = (w/\mathsf{ek}_{i,0},\ \tilde{w}/\widetilde{\mathsf{ek}}_{i,0})$, no check needed. We will show in OMUF that this is OK.] Compute $t \leftarrow \mathbb{Z}_p$; $\alpha_1, \ldots, \alpha_\ell \leftarrow \mathbb{G}_1$ subject to $\prod_i \alpha_i = \mathsf{bsk} \cdot u_0^t$ [**Ky**: attention u_0^t]
For $i \in [\ell]$, $b \in \{0,1\}$: $s_{i,b}, s'_{i,b} \leftarrow \mathbb{Z}_p$ and

$$\mathsf{ct}_{i,b} = (C_{i,b}, D_{i,b}) := \Big(g^{s_{i,b}} h^{s'_{i,b}},\ \ \mathsf{ek}_{i,b}^{s_{i,b}}\, \widetilde{\mathsf{ek}}_{i,b}^{s'_{i,b}} \cdot \alpha_i \cdot u_i^{t\cdot b}\Big).$$

$\rho_2 := \big((\mathsf{ct}_{i,b})_{i,b},\ \sigma_2 = g_2^t,\ \sigma'_2 = g_1^t\big)$.

BSDerive(st, ρ_2). Check $\mathbf{e}(\sigma'_2, g_2) = \mathbf{e}(g_1, \sigma_2)$. For $i \in [\ell]$: $\sigma_{1,i} := D_{i,M_i} \cdot C_{i,M_i}^{-y_i}$; $\sigma_1 := \prod_i \sigma_{1,i}$. Re-randomize: $t' \leftarrow \mathbb{Z}_p$,

$$\hat{\sigma}_1 := \sigma_1 \cdot \mathcal{F}(M)^{t'}, \qquad \hat{\sigma}_2 := \sigma_2 \cdot g_2^{t'}, \qquad \hat{\sigma}'_2 := \sigma'_2 \cdot g_1^{t'}.$$

Output $\sigma = (\hat{\sigma}_1, \hat{\sigma}_2, \hat{\sigma}'_2)$.

BSVerify$(\mathsf{bvk}, M, \sigma)$. Accept iff

$$\mathbf{e}(\hat{\sigma}'_2, g_2) = \mathbf{e}(g_1, \hat{\sigma}_2) \qquad \wedge \qquad \mathbf{e}(\hat{\sigma}_1, g_2) = \mathbf{e}(h_s, \mathsf{bvk}) \cdot \mathbf{e}(\mathcal{F}(M), \hat{\sigma}_2), \tag{1}$$

with $\mathcal{F}(M)$ recomputed from M.

Correctness. On the chosen branch $(\mathsf{ek}, \widetilde{\mathsf{ek}}) = (g^{y_i}, h^{y_i})$, so $\mathsf{ek}^s \widetilde{\mathsf{ek}}^{s'} = (g^s h^{s'})^{y_i} = C^{y_i}$ and $\sigma_{1,i} = \alpha_i u_i^{tM_i}$; hence $\sigma_1 = \big(\prod_i \alpha_i\big) \prod_i u_i^{tM_i} = \mathsf{bsk} \cdot u_0^t \prod_i u_i^{tM_i} = \mathsf{bsk} \cdot \mathcal{F}(M)^t$, a Waters signature with randomness t; and $\sigma'_2 = g_1^t$ passes the first check. Re-randomization preserves validity with randomness $t + t'$. Correctness is perfect.

4.2 Security of Dual-Mode OT approach

We state some notations and lemmas needed before analyzing the security (OMUF Definition 18, and blindness Definition 17) of the above blind signature [**Ky**: change to figure later, attention our blindness is honest-key].

Throughout this section fix the group $\mathbb{G}_1$ of prime order p and the OT-CRS $(g, h, w, \tilde{w}) \in \mathbb{G}_1^4$.

Definition 30 (DH pairs and messy CRS). *Assume $g \neq 1$ (so g generates $\mathbb{G}_1$) and let $\eta := \log_g h \in \mathbb{Z}_p$ (any value, including 0). A pair $(u, v) \in \mathbb{G}_1^2$ is a* DH pair *(w.r.t. (g, h)) if $v = u^\eta$. The CRS is* messy *if $g \neq 1$ and $\tilde{w} \neq w^\eta$.*

Over a uniform CRS, $\Pr[g = 1] \leq 1/p$ and $\Pr[\tilde{w} = w^\eta \mid g \neq 1] = 1/p$ (for any fixed (g, h, w) with $g \neq 1$, exactly one value of $\tilde{w}$ is bad). Hence

$$\Pr_{(g,h,w,\tilde{w})\leftarrow\mathbb{G}_1^4}[\text{CRS not messy}] \ \leq\ \tfrac{2}{p}. \tag{2}$$

Perfect lossiness of non-DH branches. We first consider the distributions of C_i, D_i from the signer [**Ky**: ref to lines in protocol once put in figure] in the case of a messy CRS.

Lemma 5 (Messy-branch lossiness). *Assume $g \neq 1$, $\eta = \log_g h$. Let $(u, v) \in \mathbb{G}_1^2$ be a* non-DH *pair, i.e. $v \neq u^\eta$. Then the random variable*

$$\big(C,\, \mathsf{mask}\big) \;:=\; \big(g^s h^{s'},\; u^s v^{s'}\big), \qquad (s, s') \leftarrow \mathbb{Z}_p^2,$$

is uniformly distributed on $\mathbb{G}_1^2$. Consequently, for every plaintext $m \in \mathbb{G}_1$, the ciphertext defined as

$$(C, D) := (g^s h^{s'},\, u^s v^{s'} \cdot m)$$

is uniform on $\mathbb{G}_1^2$, with a distribution that does not depend on m.

Proof. Since g generates $\mathbb{G}_1$, write $u = g^c$, $v = g^e$ for unique $c, e \in \mathbb{Z}_p$; the hypothesis $v \neq u^\eta$ reads $e \neq c\eta \pmod p$. Consider the $\mathbb{Z}_p$-linear map

$$\varphi : \mathbb{Z}_p^2 \to \mathbb{Z}_p^2, \qquad \varphi(s, s') = \big(s + \eta s',\; cs + es'\big), \qquad \text{with matrix } \begin{pmatrix} 1 & \eta \\ c & e \end{pmatrix},\ \det = e - c\eta.$$

As $e - c\eta \neq 0$ in the field $\mathbb{Z}_p$, φ is a bijection of $\mathbb{Z}_p^2$. Since (s, s') is uniform on $\mathbb{Z}_p^2$, so is $\varphi(s, s')$. Finally $x \mapsto g^x$ is a bijection $\mathbb{Z}_p \to \mathbb{G}_1$ (prime order, g a generator), applied coordinate-wise it maps $\varphi(s, s')$ to (C, mask), which is therefore uniform on $\mathbb{G}_1^2$. For the ciphertext statement, $D = \mathsf{mask} \cdot m$: for each fixed m, multiplication by m is a bijection of $\mathbb{G}_1$, so (C, D) is uniform on $\mathbb{G}_1^2$; in particular the distribution is the same for every m.

Corollary 1. *Let m be any $\mathbb{G}_1$-valued random variable, and let $(s, s') \leftarrow \mathbb{Z}_p^2$ be fresh, independent of m. Assume $g \neq 1$, $\eta = \log_g h$ and $(u, v) \in \mathbb{G}_1^2$ is a* non-DH *pair, i.e. $v \neq u^\eta$. Define*

$$\big(C,\, \mathsf{mask}\big) \;:=\; \big(g^s h^{s'},\; u^s v^{s'}\big), \qquad (s, s') \leftarrow \mathbb{Z}_p^2,$$

and

$$(C, D) := (g^s h^{s'},\, u^s v^{s'} \cdot m)\ .$$

If (u, v) is non-DH, then (C, D) is uniform on $\mathbb{G}_1^2$ and independent of m. *Moreover, ciphertexts formed with independent randomness pairs are jointly independent.*

Proof. By Lemma 5, the conditional distribution of (C, D) given $m = m_0$ is uniform on $\mathbb{G}_1^2$ for every m_0; a conditional distribution that does not depend on the conditioning value is independence, and the conditional argument extends verbatim to conditioning on any additional variables independent of (s, s'). Joint independence across ciphertexts follows by conditioning on all other ciphertexts' randomness. □

At most one DH branch. We now look at the existence of a DH branch.

Lemma 6 (Unique DH branch and effective message). *Assume the CRS is messy ($g \neq 1$, $\tilde w \neq w^\eta$). For any $(\mathsf{ek}_0, \widetilde{\mathsf{ek}}_0) \in \mathbb{G}_1^2$ define $(\mathsf{ek}_1, \widetilde{\mathsf{ek}}_1) := (w \cdot \mathsf{ek}_0^{-1},\ \tilde w \cdot \widetilde{\mathsf{ek}}_0^{-1})$. Then:*

(i) (Uniqueness.) *Not both $(\mathsf{ek}_0, \widetilde{\mathsf{ek}}_0)$ and $(\mathsf{ek}_1, \widetilde{\mathsf{ek}}_1)$ are DH pairs.*

(ii) (FindMessy.) *Given the trapdoor η, the deterministic algorithm $\mathsf{FindMessy}\big(\eta, (\mathsf{ek}_0, \widetilde{\mathsf{ek}}_0)\big)$ that tests $\widetilde{\mathsf{ek}}_b \stackrel{?}{=} \mathsf{ek}_b^\eta$ for $b \in \{0, 1\}$ and outputs*

$$\mu := \begin{cases} b & \text{if exactly one branch } b \text{ is DH,} \\ \bot & \text{if neither branch is DH,} \end{cases}$$

is well defined (by (i) the case "both" cannot occur).

(iii) (Honest consistency.) *If the pair was produced by the honest user with bit M_i and $y_i \in \mathbb{Z}_p$ (as in BSUser), then branch M_i is DH; hence $\mu \in \{M_i\}$ — and if additionally the other branch is non-DH (guaranteed by (i)), $\mu = M_i$ exactly.*

For a session $\rho_1 = (\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0})_{i\in[\ell]}$ *define* $\mu_i := \mathsf{FindMessy}(\eta, (\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0}))$. *The session is* usable *if* $\mu_i \neq \perp$ *for all* i*, in which case* $M^{\mathrm{eff}} := (\mu_1, \ldots, \mu_\ell) \in \{0,1\}^\ell$ *is its* effective message. *Otherwise it is* non-usable.

Proof. (i) Suppose both branches are DH: $\widetilde{\mathsf{ek}}_0 = \mathsf{ek}_0^\eta$ and $\widetilde{\mathsf{ek}}_1 = \mathsf{ek}_1^\eta$. Substituting the definitions of branch 1,

$$\tilde{w} \cdot \widetilde{\mathsf{ek}}_0^{-1} = \left(w \cdot \mathsf{ek}_0^{-1}\right)^\eta = w^\eta \cdot \mathsf{ek}_0^{-\eta} \quad \Longrightarrow \quad \tilde{w} = w^\eta \cdot \mathsf{ek}_0^{-\eta} \cdot \widetilde{\mathsf{ek}}_0 = w^\eta \cdot \mathsf{ek}_0^{-\eta} \cdot \mathsf{ek}_0^\eta = w^\eta,$$

contradicting messiness. (ii) Immediate from (i) and the definition of DH pairs, using the trapdoor η. (iii) The honest user sets $(\mathsf{ek}_{i,M_i}, \widetilde{\mathsf{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i})$, and $h^{y_i} = (g^{y_i})^\eta$, so branch M_i is DH; by (i) branch $1 - M_i$ is then non-DH, so $\mathsf{FindMessy}$ outputs exactly M_i. □

Definition 31 (Messy branch and doubly messy position). *Fix a messy OT-CRS* ($g \neq 1$, $\tilde{w} \neq w^\eta$, $\eta = \log_g h$)*. A branch key pair* $(\mathsf{ek}, \widetilde{\mathsf{ek}}) \in \mathbb{G}_1^2$ *is* messy *if it is not a DH pair, i.e.* $\widetilde{\mathsf{ek}} \neq \mathsf{ek}^\eta$*; by Lemma 5, a ciphertext formed on a messy branch with fresh* (s, s') *is uniform on* $\mathbb{G}_1^2$ *and perfectly hides its plaintext. A position* i *of a receiver message* ρ_1 *is* doubly messy *if* both *of its branches* $(\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0})$ *and* $(\mathsf{ek}_{i,1}, \widetilde{\mathsf{ek}}_{i,1}) = (w/\mathsf{ek}_{i,0}, \tilde{w}/\widetilde{\mathsf{ek}}_{i,0})$ *are messy — equivalently, if* $\mathsf{FindMessy}(\eta, (\mathsf{ek}_{i,0}, \widetilde{\mathsf{ek}}_{i,0})) = \perp$ *(Lemma 6(ii)), the case* $\mu_i = \perp$*. Thus a session is non-usable iff it contains a doubly messy position.*

Being doubly messy is a property of the (possibly adversarial) ρ_1 relative to the CRS: on a messy CRS at least one branch per position is always messy (Lemma 6(i)), honest users produce exactly one messy branch per position (Lemma 6(iii)), and at a doubly messy position both branch ciphertexts are simultaneously lossy, so the position reveals nothing about the shares α_i (hence, by Lemma 7(ii), nothing about bsk [**Ky**: this is Game 4's step]).

Remark 5. Uniqueness (i) is a property of the *CRS*, holding for *all* adversarially chosen $(\mathsf{ek}_0, \widetilde{\mathsf{ek}}_0)$. Non-usable sessions are sessions from which, by Lemma 5, the adversary learns nothing about the shares at the doubly-messy position, hence (from Lemma 7(ii) below) nothing about bsk. [**Ky**: Game 4 in OMUF makes this precise.]

Sampling during simulation. For $X \in \mathbb{G}_1$ let $\mathcal{C}_X := \{\vec{\beta} \in \mathbb{G}_1^\ell : \prod_i \beta_i = X\}$ (a coset of the subgroup $\mathcal{C}_1$). [**Ky**: Games 4 and 5 of the OMUF proof manipulate the plaintext vector $(\alpha_i\, u_i^{t\mu_i})_i$, we need the following lemma.]

Lemma 7 (Sharing lemma). *Let* $\ell \geq 1$ *and* $X \in \mathbb{G}_1$.

(i) (Sampling.) *The procedure "*$\beta_i \leftarrow \mathbb{G}_1$ *for* $i < \ell$*;* $\beta_\ell := X \cdot (\prod_{i<\ell} \beta_i)^{-1}$*" outputs a uniform element of* $\mathcal{C}_X$.

(ii) (Marginals.) *If* $\vec{\alpha}$ *is uniform on* $\mathcal{C}_X$*, then for any index* i_0 *the marginal of* $(\alpha_i)_{i\neq i_0}$ *is uniform on* $\mathbb{G}_1^{\ell-1}$ *(in particular independent of* X*).*

(iii) (Translation.) *If* $\vec{\alpha}$ *is uniform on* $\mathcal{C}_X$ *and* $\gamma_1, \ldots, \gamma_\ell \in \mathbb{G}_1$ *are fixed (or measurable with respect to conditioning variables independent of* $\vec{\alpha}$*), then* $(\alpha_i\gamma_i)_i$ *is uniform on* $\mathcal{C}_{X\cdot\prod_i \gamma_i}$.

Proof. (i) The map $(\beta_1, \ldots, \beta_{\ell-1}) \mapsto (\beta_1, \ldots, \beta_{\ell-1}, X\prod_{i<\ell} \beta_i^{-1})$ is a bijection $\mathbb{G}_1^{\ell-1} \to \mathcal{C}_X$; the uniform distribution pushes forward to the uniform distribution. (ii) By symmetry take $i_0 = \ell$ and use the parameterization of (i): under it $(\alpha_i)_{i<\ell}$ *is* the uniform seed. For general i_0, permute coordinates (the coset is permutation-invariant up to relabeling the determined coordinate). (iii) Coordinate-wise multiplication by $\vec{\gamma}$ is a bijection $\mathcal{C}_X \to \mathcal{C}_{X\prod\gamma_i}$; uniformity pushes forward. The conditional version follows by applying the statement pointwise given the conditioning. □

Remark 6 (How the lemmas are used). In the honest signer, $\vec{\alpha}$ is uniform on $\mathcal{C}_{\mathsf{bsk}\cdot u_0^t}$ (this is (i) read backwards — specify the sampling in the paper as in (i)). The DH-branch plaintext vector is $(\alpha_i\gamma_i)_i$ with $\gamma_i := u_i^{t\mu_i}$; conditioning on t and the transcript keys, (iii) says it is uniform on $\mathcal{C}_{\Sigma_1}$ with $\Sigma_1 = \mathsf{bsk}\, u_0^t \prod_i u_i^{t\mu_i} = \mathsf{bsk}\, \mathcal{F}(M^{\mathrm{eff}})^t$ — so a reduction may sample it directly from Σ_1 by (i), never touching individual u_i^t values. If some position i_0 is doubly messy, (ii) says the remaining plaintexts are uniform i.i.d. and carry no information about bsk. Corollary 1 decouples all messy-branch ciphertexts from these plaintexts.

One-more unforgeability under co-CDH We begin by adapt the partitioning techniques from [16] to our setting, and treat the "artificial abort" with respect to the simulation.

The partitioning lemma. Fix the query bound Q and set the window $m_w := 2(Q+1)$; we require $p > 2(\ell+1)m_w$ (trivially true). Sample

$$\kappa \leftarrow \{0,\dots,\ell\}, \qquad k_0,\dots,k_\ell \leftarrow \{0,\dots,m_w-1\}, \qquad j_0,\dots,j_\ell \leftarrow \mathbb{Z}_p,$$

and *program*

$$u_0 := g_1^{j_0}\,(g_1^b)^{\,k_0-\kappa m_w}, \qquad u_i := g_1^{j_i}\,(g_1^b)^{\,k_i}\ (1\le i\le \ell), \tag{3}$$

so that $\mathcal{F}(M) = g_1^{J(M)}(g_1^b)^{K(M)}$ with, *over the integers*,

$$K(M) = k_0 - \kappa m_w + \sum_{i=1}^{\ell} M_i k_i, \qquad J(M) = j_0 + \sum_{i=1}^{\ell} M_i j_i \pmod p.$$

Since $|K(M)| \le (\ell+1)m_w < p/2$, we have $K(M) \equiv 0 \pmod p$ iff $K(M) = 0$ in $\mathbb{Z}$, and $K(M)^{-1} \bmod p$ is well defined whenever $K(M) \neq 0$. [**Ky**: will move closer to the proof of OMUF later.]

Lemma 8 (Waters partitioning). *Given the above parameter set up*

(i) (Distribution.) *For every value of* $(\kappa,\vec{k})$*, the programmed* $(u_0,\dots,u_\ell)$ *of* (3) *are uniform i.i.d. on* $\mathbb{G}_1^{\ell+1}$ *(the* j_i *are uniform masks). In particular* $(\kappa,\vec{k})$ *is statistically independent of* pp.

(ii) (Programmability.) *For any* $M^* \in \{0,1\}^\ell$ *and any* $M^{(1)},\dots,M^{(Q')} \in \{0,1\}^\ell \setminus \{M^*\}$ *with* $Q' \le Q$ *(repetitions allowed),*

$$\gamma \;:=\; \Pr_{\kappa,\vec{k}}\Big[\,K(M^*)=0 \;\wedge\; \forall j\in[Q']: K(M^{(j)})\neq 0\,\Big] \;\ge\; \frac{1}{4(\ell+1)(Q+1)}.$$

Proof. (i) Fix $(\kappa,\vec{k})$ and condition on g_1^b; then $u_i = g_1^{j_i}\cdot(\text{const}_i)$ with $j_i \leftarrow \mathbb{Z}_p$ independent, and multiplication by a constant preserves uniformity; independence across i is inherited from the j_i. Independence of $(\kappa,\vec{k})$ from pp follows since the conditional law of pp given $(\kappa,\vec{k})$ does not depend on $(\kappa,\vec{k})$.

(ii) Write $S(M) := k_0 + \sum_i M_i k_i \in [0,(\ell+1)(m_w-1)]$, so $K(M) = S(M) - \kappa m_w$, and let $E^* := \{K(M^*)=0\} = \{S(M^*) = \kappa m_w\}$.

Step 1: $\Pr[E^*] = \frac{1}{(\ell+1)m_w}$*, exactly, and conditioned on* E^* *the vector* $(k_1,\dots,k_\ell)$ *remains uniform.* Fix any value of $(k_1,\dots,k_\ell)$ and let $A := \sum_i M_i^* k_i \in [0,\ell(m_w-1)]$. The event E^* requires $k_0 = \kappa m_w - A$ with $k_0 \in [0,m_w-1]$, i.e.

$$\kappa m_w - A \in [0,m_w-1] \quad\Longleftrightarrow\quad \kappa \in \Big[\tfrac{A}{m_w},\ \tfrac{A}{m_w}+1-\tfrac{1}{m_w}\Big],$$

an interval of length $1-\frac{1}{m_w} < 1$, which therefore contains *at most* one integer. It always contains *exactly* one. Write $A = qm_w + r$ with $q = \lfloor A/m_w\rfloor$ and $r = A \bmod m_w \in \{0,\dots,m_w-1\}$, and check the two cases:

- $r = 0$: the left endpoint $A/m_w = q$ is itself an integer; the unique solution is $\kappa = q$, forcing $k_0 = 0$.
- $r \ge 1$: the smallest integer $\ge A/m_w$ is $q+1$, and $q+1 \le \frac{A}{m_w}+1-\frac{1}{m_w}$ holds iff $r \ge 1$; the unique solution is $\kappa = q+1$, forcing $k_0 = m_w - r \in [1,m_w-1]$.

In both cases the unique solution is $\kappa = \lceil A/m_w\rceil$, and it lies in the sampled range $\{0,\dots,\ell\}$: it is ≥ 0 since $A \ge 0$, and $\lceil A/m_w\rceil \le \lceil \ell(m_w-1)/m_w\rceil = \lceil \ell - \ell/m_w\rceil \le \ell$ since $\ell/m_w > 0$. So for *every* fixed $(k_1,\dots,k_\ell)$, exactly one of the $(\ell+1)\cdot m_w$ equiprobable values of the independent pair (κ,k_0) realizes E^*. Hence,

$$\Pr\big[E^* \mid k_1,\dots,k_\ell\big] \;=\; \frac{1}{\ell+1}\cdot\frac{1}{m_w},$$

a constant not depending on $(k_1, \ldots, k_\ell)$. Averaging gives $\Pr[E^*] = \frac{1}{(\ell+1)m_w}$; and by Bayes, the conditional law of $(k_1, \ldots, k_\ell)$ given E^* is proportional to its (uniform) prior times a constant, i.e. uniform.

Step 2: for each j, $\Pr\big[K(M^{(j)}) = 0 \mid E^*\big] \le \frac{1}{m_w}$. On E^* we have $k_0 - \kappa m_w = -A$, hence

$$K(M^{(j)}) \;=\; S(M^{(j)}) - \kappa m_w \;=\; \sum_{i=1}^{\ell} \delta_i\, k_i, \qquad \delta_i := M_i^{(j)} - M_i^* \in \{-1, 0, 1\},$$

and $\vec{\delta} \neq \vec{0}$ because $M^{(j)} \neq M^*$. Pick i° with $\delta_{i^\circ} \neq 0$. By Step 1, given E^* the coordinates $(k_1, \ldots, k_\ell)$ are uniform i.i.d.; condition further on all k_i, $i \neq i^\circ$: the equation $\sum_i \delta_i k_i = 0$ then fixes $k_{i^\circ} = -\delta_{i^\circ}^{-1} \sum_{i \neq i^\circ} \delta_i k_i$ to a *single* integer value (recall $\delta_{i^\circ} = \pm 1$, and the arithmetic is over $\mathbb{Z}$), which the uniform $k_{i^\circ} \in \{0, \ldots, m_w - 1\}$ hits with probability at most $1/m_w$.

Step 3: combine. By a union bound over the (at most Q) indices j,

$$\gamma \;=\; \Pr[E^*] \cdot \Big(1 - \Pr\big[\exists j : K(M^{(j)}) = 0 \mid E^*\big]\Big) \;\ge\; \frac{1}{(\ell+1)m_w}\Big(1 - \frac{Q}{m_w}\Big).$$

With $m_w = 2(Q+1)$: $1 - \frac{Q}{2(Q+1)} = \frac{Q+2}{2(Q+1)} \ge \frac{1}{2}$, so $\gamma \ge \frac{1}{4(\ell+1)(Q+1)}$. □

Removing the artificial abort. Waters' original IBE [16] analysis needs an "artificial abort" because the simulator's abort probability is correlated with the adversary's queries. The success probability is the non-abort probability, approximate by a Monte-Carlo method in the original [16]. In our setting we can do better: (a) the transcript the adversary sees can be generated *without ever using* $(\kappa, \vec{k})$ except through pp, which by Lemma 8(i) is independent of them, and (b) OMUF is a *search* game — success of the reduction is the conjunction "adversary wins" ∧ "partition good", with no conditioning bias to correct, unlike IND-style games. Lemma 9 is the formalization of that folklore bound for our reduction. A systematic recent works also revisits the removal of artificial abort [9]. [**Ky**: will move closer to the proof of OMUF later, when treating Gme 6.]

Lemma 9 (Deferred sampling / coupling). *Fix the reduction $\mathcal{B}$ of Theorem 2 below and the (semantically identical) Game 6 challenger. There is a bijective correspondence between the randomness of $\mathcal{B}$'s simulation and the pair (*Game-6 randomness*, $(\kappa, \vec{k})$) under which:*

(a) the transcript of $\mathcal{B}$'s interaction with $\mathcal{A}$ (public parameters, key, all oracle answers) is identical, element by element*, to the Game-6 transcript, for as long as $\mathcal{B}$ does* not *abort; and*

(b) $\mathcal{B}$ aborts exactly on the event $\neg\mathsf{Good}$, where $\mathsf{Good} := \{\forall$ usable $j : K(M^{(j)}) \neq 0\} \wedge \{K(M^*) = 0\}$, a deterministic function of $(\kappa, \vec{k})$ and of the Game-6 transcript (which determines the effective messages $M^{(j)}$ and the target M^*).

Consequently, writing ε_6 for $\mathcal{A}$'s success probability in Game 6,

$$\Pr[\mathcal{B} \text{ outputs } g_1^{ab}] \;=\; \mathbb{E}_{\text{Game-6 run}}\Big[\mathbf{1}\{\mathcal{A} \text{ wins}\} \cdot \Pr_{\kappa,\vec{k}}\big[\mathsf{Good}\big]\Big] \;\ge\; \gamma \cdot \varepsilon_6,$$

with γ from Lemma 8(ii) (applicable since $M^* \notin S$ by construction and $|S| \le Q$).

Proof. Throughout, fix the instance exponents (a, b), where $b := \log_{g_1} h_s$ is defined information-theoretically; all statements are conditional on (a, b), and the final bound follows by averaging.

Coin spaces. Write Game 6's coins as $\omega_6 = (\omega_{\text{sh}}, \vec{u}, \vec{t}) \in \Omega_6$, where ω_{sh} collects every coin that will be accessed by $\mathcal{B}$ (e.g., the OT-CRS coins, in particular η, and, per session, the DH-branch encryption coins (s, s'), the messy-branch randoms, and the non-usable-session randoms). We define $\vec{u} = (u_0, \ldots, u_\ell) \leftarrow \mathbb{G}_1^{\ell+1}$ is the Waters vector, and $\vec{t} = (t_1, \ldots, t_Q) \leftarrow \mathbb{Z}_p^Q$ are the session randoms to answer signing queries. $\mathcal{B}$'s coin vector is $(\omega_{\text{sh}}, \kappa, \vec{k}, \vec{j}, \vec{\tilde{t}})$ with $\vec{j} \leftarrow \mathbb{Z}_p^{\ell+1}$, $\vec{\tilde{t}} \leftarrow \mathbb{Z}_p^Q$, all coordinates independent. Let $\Omega := \Omega_6 \times \mathcal{K}$ where Ω_6 contains Game 6's coins and $\mathcal{K}$ contains $(\kappa, \vec{k})$, *independent* of Ω_6.

The bijection. Define $\Phi_{a,b}\colon \Omega \to (\mathcal{B}\text{'s coin space})$ by

$$\Phi_{a,b}\big(\omega_{\mathrm{sh}}, \vec{\hat{u}}, \vec{t},\ \kappa, \vec{k}\big) := \big(\omega_{\mathrm{sh}},\ \kappa, \vec{k},\ \vec{j},\ \vec{\tilde{t}}\big), \qquad \begin{aligned} j_i &:= \log_{g_1} u_i - b\,k'_i \quad (0 \le i \le \ell),\\ \tilde{t}_j &:= t_j + f_j \qquad\qquad (1 \le j \le Q), \end{aligned}$$

where $k'_0 := k_0 - \kappa m_w$, $k'_i := k_i$ $(i \ge 1)$, and

$$f_j := \begin{cases} a \cdot K_j^{-1} \bmod p & \text{if } C_j,\\ 0 & \text{otherwise,} \end{cases} \qquad K_j := K\big(M^{(j)}\big) \text{ with } M^{(j)} \text{ is the } j\text{-th signing},$$

where $C_j := \big[\text{the } j\text{-th query exists in that run, is usable, } K_j \neq 0, \text{ and } K_{j'} \neq 0\ \forall \text{ usable } j' < j\big]$. We claim that $\Phi_{a,b}$ is well-defined.

- *(Prefix.)* $\mathcal{A}$ emits $\rho_1^{(j)}$ before t_j is used (as our blind signature is round-optimal), so the transcript up to and including the j-th query — hence session j's usability (via $\eta \in \omega_{\mathrm{sh}}$) and its effective message $M^{(j)}$ — is a deterministic function of $(\omega_{\mathrm{sh}}, \vec{\hat{u}}, t_1, \ldots, t_{j-1})$ alone. These quantities do *not* involve $(\kappa, \vec{k})$ — only K_j does, through the function K. Thus f_j depends only on coordinates *strictly preceding* t_j in the order $(\omega_{\mathrm{sh}}, \kappa, \vec{k}, \vec{\hat{u}}, t_1, \ldots, t_{j-1})$, together with (a, b) where $b := \log_{g_1} h_s$.
- *(Invertibility.)* On C_j we have $0 < |K_j| \le (\ell+1)(m_w - 1) < p/2$, so K_j is invertible mod p.
- *(Post-abort coordinates.)* If C_j fails because some earlier usable $K_{j'} = 0$, then $f_j = 0$ and $\tilde{t}_j = t_j$; $\mathcal{B}$'s run has already ended, the identity map keeps $\Phi_{a,b}$ total.

Bijectivity. $\Phi_{a,b}$ is the identity on $(\omega_{\mathrm{sh}}, \kappa, \vec{k})$. On $\vec{\hat{u}} = (u_0, u_1, \ldots, u_\ell)$ from (3) (Lemma 8(i)) by construciton of $\Phi_{a,b}$, $j_i := \log_{g_1} u_i - b\,k'_i$, where $k'_0 := k_0 - \kappa m_w$ and $k'_i := k_i$ $(i \ge 1)$ that recovers the exponents of u_i. Consequently, it is, for each fixed $(b, \kappa, \vec{k})$, the discrete-log identification $\mathbb{G}_1 \cong \mathbb{Z}_p$ followed by the translation by $-b\,k'_i$ — a bijection carrying uniform random to uniform random, and precisely the inverse of the programming (3). On $\vec{t}$ it is *triangular*: the offset f_j is a function of strictly earlier coordinates. The inverse is computed coordinate by coordinate in the stated order: $u_i = g_1^{j_i + b k'_i}$ then, for $j = 1, 2, \ldots$, evaluate f_j on the already-recovered prefix and set $t_j = \tilde{t}_j - f_j$. Every $(t, \kappa, \vec{k})$ map is a translation of $\mathbb{Z}_p$ (or the log-identification of $\mathbb{G}_1$), so each conditional law given the preceding coordinates is carried uniform-to-uniform. By the chain rule for product probability measures, $\Phi_{a,b}$ preserves the uniform distribution of Ω to exactly the uniform distribution of $\mathcal{B}$'s coin space. The two consequences are:*(1)* $\mathcal{B}$'s coins, thus its simulation, are distributed exactly as prescribed and *(2)* $(\kappa, \vec{k})$ is independent of the Game-6 run — it may be deferred and sampled only after that run completes.

(a) Transcript equality by induction on sessions. On the event that $K(M^{(j')}) \neq 0$ for every usable session $j' < j$ (all quantities read off the *Game-6* run), $\mathcal{B}$ has not aborted before session j, and the two transcripts agree up to session $j-1$ — in particular session j's $\rho_1^{(j)}$, usability and $M^{(j)}$ agree.

- Base case: setup coincides (pp by the $\vec{\hat{u}} \leftrightarrow \vec{j}$ identification; $\mathsf{bvk} = g_2^a = A_2$; $h_s = g_1^b = B$; OT-CRS shared).
- Induction step, session j: by hypothesis the prefixes agree, so $\mathcal{A}$ (same tape, same prefix) sends the same $\rho_1^{(j)}$, and $\mathsf{FindMessy}$ (shared η) yields the same usability status and effective message. Three cases:
  - *Non-usable:* both experiments answer with fresh shared uniforms and a shared self-chosen t; identical answers, no $(\kappa, \vec{k})$-dependence.
  - *Usable, $K_j \neq 0$:* here C_j holds, so $\tilde{t}_j = t_j + a/K_j$, and $\mathcal{B}$'s simulation values equal Game 6's values:

$$\begin{aligned} \Sigma_1^{(j)} &= (g_1^a)^{-J_j/K_j}\, \mathcal{F}(M^{(j)})^{\tilde{t}_j} = (g_1^a)^{-J_j/K_j} \cdot g_1^{(bK_j + J_j)(t_j + a/K_j)}\\ &= (g_1^a)^{-J_j/K_j} \cdot \mathcal{F}(M^{(j)})^{t_j} \cdot g_1^{ab} \cdot (g_1^a)^{J_j/K_j} = \mathsf{bsk} \cdot \mathcal{F}(M^{(j)})^{t_j},\\ \sigma_2^{(j)} &= g_2^{\tilde{t}_j} (g_2^a)^{-1/K_j} = g_2^{t_j}, \qquad \sigma_2'^{(j)} = g_1^{\tilde{t}_j} (g_1^a)^{-1/K_j} = g_1^{t_j}. \end{aligned}$$

When responding the signing queries, the sharing of $\Sigma_1^{(j)}$, when computing $t \leftarrow \mathbb{Z}_p$ then share $\alpha_1, \ldots, \alpha_\ell \leftarrow \mathbb{G}_1$ subject to $\prod_i \alpha_i = \mathsf{bsk} \cdot u_0^t$, uses the *same* coset representative $\Sigma_1^{(j)}$ (applying Lemma 7(i)). The DH-branch encryptions of the α_i are computable from the received key elements and fresh shared (s, s'), and the uniform messy branches then coincide element by element.

- *Usable, $K_j = 0$:* $\mathcal{B}$ aborts at session j, per the definition of $f_{j'}$, all later coordinate maps are the identity.

This proves (a). Moreover the induction shows $\mathcal{B}$'s abort time is exactly the first usable Game-6 session with $K = 0$, and the final abort condition $K(M^*) \neq 0$ is evaluated on the shared forgeries.

(b) Abort events: Good is a deterministic function of $(\kappa, \vec{k})$ and of the Game-6 run alone, since the effective messages $M^{(j)}$, the multiset S, the win/lose status, and M^* (first output message outside S in the fixed output ordering) are all functions of the Game-6 transcript, which never touches $(\kappa, \vec{k})$.

The bound. On $\mathsf{Good} \cap \{\mathcal{A} \text{ wins}\}$, no abort occurs, the runs coincide in full, and [**Ky**: Theorem 2, extraction step in Game 6] $\mathcal{B}$ outputs g_1^{ab} with certainty. Hence, by independence of $\mathcal{K}$ from Ω_6,

$$\Pr[\mathcal{B} \text{ outputs } g_1^{ab}] \;\geq\; \Pr_{\Omega}\big[\mathsf{Good} \wedge \mathcal{A} \text{ wins}\big] \;=\; \mathbb{E}_{\Omega_6}\Big[\mathbf{1}\{\mathcal{A} \text{ wins}\} \cdot \Pr_{\mathcal{K}}\big[\mathsf{Good}\big]\Big],$$

and Lemma 8(ii) applies *pointwise* to each winning run: its target M^* differs from every usable effective message (not in S), the usable effective messages number $Q' \leq Q$, so $\Pr_{\mathcal{K}}[\mathsf{Good}] \geq \gamma$. □

Remark 7. Our loss $4(\ell+1)(Q+1)$ at time $T + O(\ell Q)$ has the profile $(\, O(\varepsilon/(\ell Q)),\, T + O(\ell Q)\,)$ that is ideal for partitioning-based proofs, as the $\Theta(Q)$ factor itself is unavoidable for Waters-type signatures by the black-box lower bound of Hofheinz–Jäger–Knapp [10].

4.3 The theorem

Theorem 2 (One-more unforgeability of Π_A). *For every PPT adversary $\mathcal{A}$ making at most Q queries in the experiment of Definition 18, there is an algorithm $\mathcal{B}$, running $\mathcal{A}$ once plus* $\mathsf{poly}(\lambda, \ell, Q)$ *group operations, such that*

$$\mathsf{Adv}^{\mathrm{omuf}}_{\mathcal{A}}(\lambda) \;\leq\; 4(\ell+1)(Q+1) \cdot \mathsf{Adv}^{\mathsf{co\text{-}CDH}}_{\mathcal{B}}(\lambda) \;+\; \frac{2}{p}.$$

Proof. Let ε_i denote $\mathcal{A}$'s success probability in Game i. Game 0 is the real experiment of Definition 18 with Π_A, in which the challenger knows every discrete log it samples (in particular η and a).

Game 1 (CRS sanity). Abort if the OT-CRS is not messy, i.e. if $g = 1$ or $\tilde{w} = w^\eta$. By (2), $|\varepsilon_0 - \varepsilon_1| \leq 2/p$. From now on Definitions 30 and Lemma 6 apply.

Game 2 (find messy branches). On each query ρ_1, compute $(\mu_i)_i$ and, if usable, M^{eff}, via $\mathsf{FindMessy}(\eta, \cdot)$ (Lemma 6(ii)). Purely syntactic: $\varepsilon_2 = \varepsilon_1$.

Game 3 (messy branches go uniform). In every session, for every position i and branch b with $(\mathsf{ek}_{i,b}, \widetilde{\mathsf{ek}}_{i,b})$ non-DH, replace $\mathsf{ct}_{i,b}$ by a fresh uniform pair in $\mathbb{G}_1^2$. Each such ciphertext uses its own fresh $(s_{i,b}, s'_{i,b})$ appearing nowhere else, so by Lemma 5 and Corollary 1 the replaced ciphertexts were already uniform and *jointly independent* of everything else — including the correlated plaintexts $\alpha_i u_i^{tb}$ and the DH-branch ciphertexts. $\varepsilon_3 = \varepsilon_2$ (perfect).

Game 4 (non-usable sessions). In every session with some $\mu_{i_0} = \perp$, replace additionally *all* remaining (DH-branch) plaintexts by independent uniform elements of $\mathbb{G}_1$. We keep $\sigma_2 = g_2^t, \sigma'_2 = g_1^t$ with an honestly sampled t. The transition is perfect: after Game 3, fixing i_0 in this game where $\mu_{i_0} = \perp$, the share α_{i_0} appears in no ciphertext component the transcript still depends on. Hence by Lemma 7(ii) the $(\alpha_i)_{i \neq i_0}$ is uniform i.i.d. and independent of (bsk, t). Multiplying coordinate-wise $(\alpha_i)_{i \neq i_0}$ by the offsets $u_i^{t\mu_i}$ preserves uniform i.i.d. (Lemma 7(iii), applied conditionally on t and the keys). As consequence, the replaced plaintexts have exactly the honest conditional distribution: $\varepsilon_4 = \varepsilon_3$.

Game 5 (usable sessions re-parameterized). In every usable session with effective message $M^{(j)}$ and signer randomness t_j:

– compute $\Sigma_1^{(j)} := \mathsf{bsk} \cdot \mathcal{F}(M^{(j)})^{t_j}$;
– sample $\vec{\beta}$ uniform on $\mathcal{C}_{\Sigma_1^{(j)}}$ via Lemma 7(i);
– set the DH-branch ciphertexts to honest encryptions of the β_i, i.e. $D_{i,\mu_i} := \mathsf{ek}_{i,\mu_i}^{s}\widetilde{\mathsf{ek}}_{i,\mu_i}^{s'}\beta_i$ with fresh (s, s') — computable directly from the received key elements by the adversary's first flow.

Perfect transition: honestly, the DH-branch plaintext vector is $(\alpha_i u_i^{t_j\mu_i})_i$ with $\vec{\alpha}$ uniform on $\mathcal{C}_{\mathsf{bsk}u_0^{t_j}}$; by Lemma 7(iii) (conditionally on t_j and the keys) this vector is uniform on $\mathcal{C}_{\Sigma_1^{(j)}}$ — exactly the law of $\vec{\beta}$. Independence from the messy branches is Corollary 1. $\varepsilon_5 = \varepsilon_4$.

Game 6 (partition sampling, reduction to co-CDH). Let S be the *multiset* of effective messages of usable sessions, $|S| \le Q$. When $\mathcal{A}$ wins, i.e. outputs $Q{+}1$ verifying pairs with pairwise-distinct *messages* (Definition 18), at least one output message lies outside the *support* of S. Let M^* be the first such message in a fixed ordering of the output list, which is a deterministic function of the transcript. *After* $\mathcal{A}$ halts, the challenger draws fresh coins $(\kappa, \vec{k}) \leftarrow \mathcal{K}$, uniform and independent of the entire run, and declares Game 6 *won* iff

$$\mathcal{A} \text{ wins } \wedge\ \mathsf{Good}, \qquad \mathsf{Good} := \big\{\forall \text{ usable } j:\ K(M^{(j)}) \neq 0\big\} \wedge \big\{K(M^*) = 0\big\}.$$

Since $(\kappa, \vec{k})$ is drawn independently of the run,

$$\varepsilon_6 \ =\ \mathbb{E}\Big[\mathbf{1}\{\mathcal{A} \text{ wins}\} \cdot \Pr_{\kappa,\vec{k}}\big[\mathsf{Good}\big]\Big] \ \ge\ \gamma \cdot \varepsilon_5, \qquad \gamma \ \ge\ \tfrac{1}{4(\ell+1)(Q+1)},$$

by Lemma 8(ii) applied *pointwise* to each winning run: its target M^* avoids every usable effective message ($M^* \notin \operatorname{supp} S$, part (i)) and the usable sessions number $Q' \le Q$.

Simulation from a co-CDH instance. Each earlier hop removed one use of a secret, and the reduction below consumes exactly what is left. Game 1 guarantees a messy CRS, so $\mathsf{FindMessy}$ (Lemma 6) is total and Game 2's usability and effective messages are a computation from η that $\mathcal{B}$ performs itself. Game 3 filled every messy branch with fresh uniforms, so $\mathcal{B}$ never needs the off-branch plaintexts (which involve u_i^t values it cannot compute). Game 4 made non-usable sessions independent of bsk and of the Waters exponents, so $\mathcal{B}$ answers them from public values alone. Game 5 compressed each usable session's secret-dependence into the *single* element $\Sigma_1^{(j)} = \mathsf{bsk} \cdot \mathcal{F}(M^{(j)})^{t_j}$ — precisely the element the Waters trick works from (g_1^a, g_1^b) when $K(M^{(j)}) \neq 0$. Because every hop from Game 2 onward is *perfect*, the coupling of Lemma 9 can be applied. The only remaining gap between Game 6 and the reduction is *when* the partition coins are drawn: at the end of the run (Game 6) versus before setup ($\mathcal{B}$, which needs them to program pp and to answer queries). Via the below simulation by $\mathcal{B}$, Lemma 9 shows identical joint distributions with Game 6, and this concludes the reduction to co-CDH.

The reduction $\mathcal{B}(g_1, g_2, A_1{=}g_1^a, A_2{=}g_2^a, B{=}g_1^b)$ simulates Game 6:

– *Setup/keys.* OT-CRS honestly (knows η; abort as in Game 1); $h_s := B$; $\mathsf{bvk} := A_2$ (so $\mathsf{bsk} = g_1^{ab}$ is the co-CDH answer); Waters vector programmed as (3) with $m_w = 2(Q+1)$. Distributions are identical by Lemma 8(i).
– *Oracle, non-usable sessions:* as in Game 4 (uniform components, self-chosen t); no secrets needed.
– *Oracle, usable sessions:* compute $M^{(j)}$ via $\mathsf{FindMessy}$; if $K(M^{(j)}) = 0$, abort. Else $\tilde{t}_j \leftarrow \mathbb{Z}_p$ and

$$\Sigma_1^{(j)} := A_1^{-J(M^{(j)})/K(M^{(j)})} \cdot \mathcal{F}(M^{(j)})^{\tilde{t}_j}, \qquad \sigma_2^{(j)} := g_2^{\tilde{t}_j} A_2^{-1/K(M^{(j)})}, \qquad \sigma_2'^{(j)} := g_1^{\tilde{t}_j} A_1^{-1/K(M^{(j)})},$$

then exactly as Game 5 ($\vec{\beta}$ uniform on $\mathcal{C}_{\Sigma_1^{(j)}}$, honest DH-branch encryptions, uniform messy branches). By Lemma 9 these are *the same group elements* as Game 6's.
– *Finish.* On $\mathcal{A}$'s output $\{(m_i, \sigma_i)\}_{i=1}^{Q+1}$: verify all pairs and pairwise-distinctness; determine M^* as in Game 6; abort if $K(M^*) \neq 0$. Otherwise *extraction*: write $\sigma^* = (\hat{\sigma}_1^*, \hat{\sigma}_2^*, \hat{\sigma}_2'^*)$ for the forgery on M^*.
  • The first equation of (1) forces a *common exponent*: writing $\hat{\sigma}_2^* = g_2^{t^*}$ and $\hat{\sigma}_2'^* = g_1^{t^{**}}$ for some unknown well-defined t^*, t^{**}. Then $\mathbf{e}(\hat{\sigma}_2'^*, g_2) = \mathbf{e}(g_1, \hat{\sigma}_2^*)$ reads $\mathbf{e}(g_1, g_2)^{t^{**}} = \mathbf{e}(g_1, g_2)^{t^*}$, and $\mathbf{e}(g_1, g_2)$ generates $\mathbb{G}_\mathsf{t}$ (non-degeneracy, prime order), so $t^{**} = t^*$.

- The second equation forces $\hat{\sigma}_1^* = \mathsf{bsk} \cdot \mathcal{F}(M^*)^{t^*}$: indeed $\mathbf{e}(\hat{\sigma}_1^*, g_2) = \mathbf{e}(h_s, \mathsf{bvk})\mathbf{e}(\mathcal{F}(M^*), \hat{\sigma}_2^*) = \mathbf{e}(\mathsf{bsk}\,\mathcal{F}(M^*)^{t^*}, g_2)$, and $x \mapsto \mathbf{e}(x, g_2)$ is injective on $\mathbb{G}_1$ (by the fact that $\mathbf{e}(g_1^z, g_2) = 1 \Leftrightarrow z = 0$).
- Since $K(M^*) = 0$, $\mathcal{F}(M^*) = g_1^{J(M^*)}$ and thus $\hat{\sigma}_1^* = g_1^{ab} \cdot g_1^{J(M^*)\,t^*} = g_1^{ab} \cdot (\hat{\sigma}_2'^*)^{J(M^*)}$. Output

$$Z \;:=\; \hat{\sigma}_1^* \cdot \left(\hat{\sigma}_2'^*\right)^{-J(M^*)} \;=\; g_1^{ab}.$$

Collecting the hops: $\varepsilon_0 \le \varepsilon_1 + 2/p$ (Game 1, CRS sanity) and $\varepsilon_1 = \varepsilon_2 = \varepsilon_3 = \varepsilon_4 = \varepsilon_5$ (perfect hops, Lemmas 5–7), then $\varepsilon_6 \ge \gamma\,\varepsilon_5$ with $\gamma \ge 1/(4(\ell+1)(Q+1))$ (Game 5 to 6, the only lossy hop, via Lemma 8(ii), and Lemma 9 applied on the reduction of $\mathcal{B}$ as above), finally $\mathsf{Adv}^{\mathsf{co\text{-}CDH}}_{\mathcal{B}} \ge \varepsilon_6$. Hence

$$\mathsf{Adv}^{\mathrm{omuf}}_{\mathcal{A}} \;=\; \varepsilon_0 \;\le\; \varepsilon_5 + \tfrac{2}{p} \;\le\; \tfrac{1}{\gamma}\,\varepsilon_6 + \tfrac{2}{p} \;\le\; 4(\ell+1)(Q+1) \cdot \mathsf{Adv}^{\mathsf{co\text{-}CDH}}_{\mathcal{B}} + \tfrac{2}{p}.$$

$\mathcal{B}$ runs $\mathcal{A}$ once; its overhead is $O(\ell)$ exponentiations per session plus setup. □

References

1. Abe, M., Ambrona, M., Bogdanov, A., Ohkubo, M., Rosen, A.: Acyclicity programming for sigma-protocols. In: Nissim, K., Waters, B. (eds.) TCC 2021, Part I. LNCS, vol. 13042, pp. 435–465. Springer, Heidelberg (Nov 2021). https://doi.org/10.1007/978-3-030-90459-3_15
2. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs. In: Kiltz, E., Vaikuntanathan, V. (eds.) TCC 2022, Part I. LNCS, vol. 13747, pp. 113–142. Springer, Heidelberg (Nov 2022). https://doi.org/10.1007/978-3-031-22318-1_5
3. Bellare, M., Fuchsbauer, G., Scafuro, A.: NIZKs with an untrusted CRS: Security in the face of parameter subversion. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part II. LNCS, vol. 10032, pp. 777–804. Springer, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53890-6_26
4. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 390–399. ACM Press (Oct / Nov 2006). https://doi.org/10.1145/1180405.1180453
5. Couteau, G., Goudarzi, D., Klooß, M., Reichle, M.: Sharp: Short relaxed range proofs. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 609–622. ACM Press (Nov 2022). https://doi.org/10.1145/3548606.3560628
6. Couteau, G., Peters, T., Pointcheval, D.: Removing the strong RSA assumption from arguments over the integers. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 321–350. Springer, Heidelberg (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_11
7. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. In: Blakley, G.R., Chaum, D. (eds.) CRYPTO'84. LNCS, vol. 196, pp. 10–18. Springer, Heidelberg (Aug 1984)
8. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO'86. LNCS, vol. 263, pp. 186–194. Springer, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
9. Hanaoka, G., Katsumata, S., Kimura, K., Takemure, K., Yamada, S.: Tighter adaptive IBEs and VRFs: Revisiting waters' artificial abort. In: Theory of Cryptography Conference 2024 (2024), https://eprint.iacr.org/2024/1481
10. Hofheinz, D., Jager, T., Knapp, E.: Waters signatures with optimal security reduction. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012. LNCS, vol. 7293, pp. 66–83. Springer, Heidelberg (May 2012). https://doi.org/10.1007/978-3-642-30057-8_5
11. Juels, A., Luby, M., Ostrovsky, R.: Security of blind digital signatures (extended abstract). In: Kaliski Jr., B.S. (ed.) CRYPTO'97. LNCS, vol. 1294, pp. 150–164. Springer, Heidelberg (Aug 1997). https://doi.org/10.1007/BFb0052233
12. Katsumata, S., Reichle, M., Sakai, Y.: Practical round-optimal blind signatures in the rom from standard assumptions. to appear in Asiacrypt (2023), https://eprint.iacr.org/2023/1447, https://eprint.iacr.org/2023/1447
13. Peikert, C., Vaikuntanathan, V., Waters, B.: A framework for efficient and composable oblivious transfer. In: Wagner, D. (ed.) CRYPTO 2008. LNCS, vol. 5157, pp. 554–571. Springer, Heidelberg (Aug 2008). https://doi.org/10.1007/978-3-540-85174-5_31
14. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. Journal of Cryptology **13**(3), 361–396 (Jun 2000). https://doi.org/10.1007/s001450010003
15. Reichle, M., Reinke, Z.: Threshold blind signatures from CDH. In: Public-Key Cryptography 2026 (2025), https://eprint.iacr.org/2025/1798

16. Waters, B.R.: Efficient identity-based encryption without random oracles. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 114–127. Springer, Heidelberg (May 2005). https://doi.org/10.1007/11426639_7

Supplementary Material